Easy Explanation

JAVASCRIPT

YOU NEED FOR INTERVIEW

SYED SHAAZ AKHTAR

JavaScript

JavaScript ek programming language hai jo tumhari website ko interactive banata hai. Matlab, agar HTML ek skeleton hai aur CSS uska design hai, toh JavaScript us skeleton ko jaan deta hai.

Jaise ki tumhe button click karna ho aur kuch actions dikhana ho, ya kisi input field mein type karte hi suggestions aayein—ye sab JavaScript ka kamaal hai!

Weakly Typed Language

JavaScript ko weakly typed language kehte hain. Matlab, tum ek hi variable mein alag-alag type ke data store kar sakte ho.

j

```
let name = 'Shaaz';
// Yeh ek string hai.


name = 5;
// Ab yeh ek number ban gaya.


name = { age: 23 };
// Ab yeh ek object ban gaya.
```

JavaScript tumse strict rules follow karne ko nahi bolta, isliye beginner-friendly hai.

Console ka Magic

JavaScript ka output screen par dekhne ke liye hum **console.log()** use karte hain. Yeh tumhare browser ke "console" section mein result dikhata hai.

example:

```
let name = "Shaaz";
console.log(name);
// Output: Shaaz
```

Console kaise open karein?

1. Browser kholo (Chrome recommended).
2. Right-click karo -> "Inspect" par click karo -> "Console" tab pe jao.

Variables - Data Store Karne Ka Dibba

Variable ek container hai jisme tum data store kar sakte ho.

JavaScript mein 3 tareeke hain variables declare karne ke liye:

1. **var:** Purana tareeka, overwrite kar sakte ho.
2. **let:** Naya tareeka, block ke andar limited rehta hai.
3. **const:** Permanent value, badal nahi sakte.

Example:

```
var score = 10;
let name = "Shaaz";
const pi = 3.14;
```

Data Types - Kya-Kya Cheezein Store Hoti Hain?

JavaScript ke paas alag-alag type ka data store karne ke options hain:

Numbers: Jaise 1, 2, 3.14.

let age = 25;

Strings: Text ke liye double ya single quotes.

let name = "Shaaz";

Booleans: True ya False.

let isLoggedIn = true;

null: Jab tum intentionally kuch khaali store karna chaho.

let emptyValue = null;

undefined: Jab tumne variable declare kiya hai, par value assign nahi ki.

let noValue;

console.log(noValue); // undefined

Objects: Key-value pairs ke liye.

let person = { name: "Shaaz", age: 25 };

Arrays: List of items.

let fruits = ["Apple", "Banana", "Mango"];

Equality - Compare

JavaScript mein tum values ko compare kar sakte ho:

== (Loose Equality): Bas values check karta hai, types nahi.

console.log(5 == "5"); // true

=== (Strict Equality): Values aur types dono check karta hai.

console.log(5 === "5"); // false

Conditional Statements - Agar Ye, Toh Wo

Kuch conditions check karke, alag-alag code chalana ho, toh if-else ka use karte hain.

Example:

```
let age = 18;


if (age >= 18) {
  console.log("You can vote!");
} else {
  console.log("Sorry, you're too young to vote.");
}
```

Logical Operators - Conditions Ko Jodo

Agar tumhe ek condition se zyada check karni hai, toh logical operators ka use hota hai:

&& (AND): Dono condition true honi chahiye.

```
if (age > 12 && age < 20) {
  console.log("You're a teenager!");
}
```

|| (OR): Koi ek condition true ho toh chalega.

```
if (age === 18 || age === 19) {
  console.log("You're just starting adulthood!");
}
```

Loops - Repeat Karna

Kabhi-kabhi tumhe ek kaam baar-baar karna hota hai, tab loops kaam aate hain.

Example:

```
for (let i = 1; i <= 5; i++) {
  console.log("JavaScript rocks!");
}
```

String Concatenation - Strings Ko Jodo

Strings aur values ko jodne ke liye + operator ka use hota hai.

Example:

```
let name = "Shaaz";

console.log("Hello, " + name + "!");

// Output: Hello, Shaaz!
```

Strings

Strings ko Samajhna aur Declare karna

String is basically ek sequence of characters.

Isse hum single quotes ' ', double quotes " ", ya backticks ke beech likhte hain.

```
let firstName = 'Shaaz'; // Single quotes
let greeting = "Hello";  // Double quotes
let intro = `My name is ${firstName}`;
// Template Literal (advanced)
console.log(intro);
// Output: My name is Shaaz
```

Strings Print Karna

```
let name = "Shaaz";
console.log("Hello " + name); // Concatenation with +
console.log(`Hello ${name}`); // Concatenation with Template Literal
```

💡 Pro Tip: Template literals use backticks aur expressions ko ${ } mein likhte hain. Yeh simple aur readable hota hai.

String Concatenation (Joining Strings)

Method 1: + Operator

```
let firstName = "Hello";
let lastName = "World";
console.log(firstName + " " + lastName); // Output: Hello World
```

Method 2: Template Literals

```
let name = "Shaaz";
console.log(`Welcome, ${name}!`); // Output: Welcome, Shaaz!
```

Characters ko Access Karna

Strings are like arrays of characters, toh index use karke hum specific character ko access karte hain:

```
let word = "JavaScript";
console.log(word[0]); // Output: J (index starts at 0)
console.log(word.charAt(4));
 // Output: S (character at index 4)
```

Case Conversion (Uppercase/Lowercase)

Kabhi kisi input ka case change karna ho, toh yeh methods kaam aate hain:

```
let sentence = "Coding is FUN!";
console.log(sentence.toLowerCase()); // Output: coding is fun!
console.log(sentence.toUpperCase()); // Output: CODING IS FUN!
```

Strings Trim Karna (Unnecessary Spaces Hataana)

Kabhi string ke starting ya ending spaces ko hatana ho, toh trim() use karte hain:

```
let messyString = "   Hello World!   ";
console.log(messyString.trim()); // Output: Hello World!
```

Substring Slicing (Ek Part Extract Karna)

Hum kisi string ka specific part extract kar sakte hain:

```
let text = "JavaScript";
console.log(text.slice(0, 4)); // Output: Java (index 0 se 3 take)
```

Splitting Strings (Parts Mein Todna)

String ko parts mein todne ke liye split() use karte hain:

```
let colors = "red,green,blue";
console.log(colors.split(',')); // Output: ["red", "green", "blue"]
```

String Search (Finding Characters)

indexOf(): Pehli baar kisi character ya word ka index batata hai.

```
let word = "programming";
console.log(word.indexOf('g')); // Output: 3
```

lastIndexOf(): Last occurrence ka index batata hai.

```
console.log(word.lastIndexOf('g')); // Output: 10
```

includes(): Check karta hai ki string mein koi word ya character hai ya nahi.

```
console.log(word.includes('gram')); // Output: true
```

Repeating Strings

```
console.log("ha".repeat(3)); // Output: hahaha
```

String Mutability

Strings immutable hoti hain, iska matlab unhe directly modify nahi kar sakte. But variable ko nayi value assign kar sakte ho:

```
let str = "Hello";
str = "Shaaz";
console.log(str); // Output: Shaaz
```

Advanced Methods

startsWith() and endsWith()

Yeh check karte hain ki string kisi specific character ya word se shuru/end hoti hai ya nahi.

```
let word = "JavaScript";
console.log(word.startsWith("Java")); // Output: true
console.log(word.endsWith("Script")); // Output: true
```

concat()

Strings ko join karne ka ek aur tareeka:

```
let part1 = "Hello";
let part2 = "World";
console.log(part1.concat(" ", part2)); // Output: Hello World
```

JavaScript Functions

Functions Kya Hote Hain?

Ek function ek block of code hota hai jo ek specific kaam karne ke liye likha jaata hai. Functions code ko modular aur reusable banate hain.

Function Declarationfunction greet() {

console.log("Hello, Shaaz!"); // Task 1

console.log("Kaise ho?"); // Task 2

}

Function Call greet();

// Output:Hello, Shaaz!

Kaise ho?

Explanation:

1. **function greet()** → Yeh function define kar raha hai.
2. **greet();** → Yeh function ko call kar raha hai, yani code ko execute kar raha hai.

Function Parameters aur Arguments

Parameters wo placeholders hote hain jo function definition mein use hote hain, aur arguments wo actual values hain jo function ko call karte waqt pass karte hain.

Example:

```
function greet(name) {
// 'name' is the parameter
console.log(`Hello, ${name}!`);
}

greet("Shaaz"); // "Shaaz" is the argument
// Output: Hello, Shaaz!

greet("Rahul");
// Output: Hello, Rahul!
```

Function with Return Statement

Functions me hum return ka use karte hain output ko function ke bahar le jaane ke liye.

```
function square(x) {
    return x * x;
  // x ko square karke return karega
}
let result = square(5); // Result store hogaya
console.log(result); // Output: 25
```

Tip: Agar return nahi likha, toh function automatically undefined return karega.

Function Expressions

Functions ko variables mein store karke bhi use kar sakte hain.

```
let greet = function(name) {
    return `Hello, ${name}!`;
};
console.log(greet("Shaaz"));
// Output: Hello, Shaaz!
```

Default Parameter Values

Agar koi argument pass na kare, toh hum default values set kar sakte hain.

```
function greet(name = "Guest") {
    console.log(`Hello, ${name}!`);
}

greet(); // Output: Hello, Guest!
greet("Shaaz"); // Output: Hello, Shaaz!
```

Function Without Parameters

Agar function me koi parameter na ho, toh bhi hum tasks perform kar sakte hain.

```
function sayHi() {
    console.log("Hello, World!");
}

sayHi();  // Output: Hello, World!
```

Passing Arrays as Arguments

Arrays ko arguments ke roop me pass kar sakte hain:

```
function sumOfTwoNumbers(numbers) {
   return numbers[0] + numbers[1];
}

let myNumbers = [10, 20];
console.log(sumOfTwoNumbers(myNumbers));
// Output: 30
```

Functions with Unlimited Parameters

Agar hume unknown number of arguments pass karne hain, toh hum arguments object use karte hain.

```
function sumAll() {
   let sum = 0;
   for (let i = 0; i < arguments.length; i++) {
      sum += arguments[i];
   }
   return sum;
}

console.log(sumAll(1, 2, 3, 4, 5));
// Output: 15
```

Arrow Functions

Arrow functions short syntax provide karte hain:

Example 1: Single Statement

```
const square = (x) => x * x;

console.log(square(5));

// Output: 25
```

Example 2: Multiple Statements

```
const greet = (name) => {
   console.log(`Hello, ${name}!`);
   return `How are you, ${name}?`;
};

console.log(greet("Shaaz"));

// Output:

Hello, Shaaz!

How are you, Shaaz?
```

Example 3: Object Return

```
const calculate = (x, y) => ({ sum: x + y, diff: x - y });

console.log(calculate(7, 3));

// Output: { sum: 10, diff: 4 }
```

JavaScript Arrays

Ek array ek box jaisa hai, jisme hum multiple cheezein (values) ek saath store kar sakte hain. Values alag-alag types ki ho sakti hain—numbers, strings, booleans, objects, aur even doosre arrays!

Why do we need Array ?

1. **Group Similar Data**: Ek saath similar cheezon ko store karna easy ho jata hai.
2. **Handle Data**: Bahut saara data ek variable mein store kar sakte ho.
3. **Enhace Performance**: JavaScript arrays ko handle karne ke liye optimize hai.
4. **Convenient methods**: Array ke saath kaam karne ke liye bahut saare built-in methods milte hain.
5. **Readable code**: Code dekhne aur samajhne mein asaan lagta hai.

How to make an Array ?

Square Brackets Method

```
let arr = [1, 2, 3, "Shaaz", false];
console.log(arr);
// Output: [1, 2, 3, "Shaaz", false]
```

Simple! Square brackets ke andar values likho, bas array ban gaya.

Array Constructor Method

```
let arr2 = new Array(1, 2, 3, "Shaaz", false);
console.log(arr2);
// Output: [1, 2, 3, "Shaaz", false]
```

Par, zyada log square brackets wala method use karte hain kyunki yeh short aur clean hota hai.

How to Access Array Element?

Array ke har element ka ek index hota hai, jo 0 se start hota hai.

```
let arr3 = ["Shaaz", "Laptop", 23, "Mobile"];

// Ek specific index ka value lena
   console.log(arr3[2]);
// Output: 23 (index 2 ka element)
    arr3[3] = 78;
// Ek element ko change kar diya
    console.log(arr3);
// Output: ["Shaaz", "Laptop", 23, 78]// index doesn't exist
   console.log(arr3[8]);
// Output: undefined
```

Common Array Methods

1. push() aur pop()

push() se array ke end mein element add hota hai.

pop() se array ka last element remove ho jata hai.

```
let newArr = [1, 2, 3, 4, 5];

newArr.push(29); // End mein add kiya
console.log(newArr);
// Output: [1, 2, 3, 4, 5, 29]

newArr.pop(); // Last element hata diya
console.log(newArr);
// Output: [1, 2, 3, 4, 5]
```

2. shift() aur unshift()

shift() se first element remove hota hai.

unshift() se array ke start mein element add karte hain.

```
newArr.shift(); // Pehla element hata diya
console.log(newArr);
// Output: [2, 3, 4, 5]

newArr.unshift(99, 100); // Starting mein elements add kiye
console.log(newArr);
// Output: [99, 100, 2, 3, 4, 5]
```

3. concat()

Do ya zyada arrays ko merge karne ke liye.

```
let a1 = [2, 3, 4, 5];

let a2 = [9, 12, 15];

let a3 = a1.concat(a2);

console.log(a3);

// Output: [2, 3, 4, 5, 9, 12, 15]
```

4. join()

Array ke elements ko ek string mein convert karta hai, specified separator ke saath.

```
let a4 = a3.join("");

console.log(a4);

// Output: "234591215"

let a5 = a3.join("@");

console.log(a5);

// Output: "2@3@4@5@9@12@15"
```

5. reverse()

Array ke elements ka order ulta kar deta hai.

```
let arr6 = [88, 22, 34, 12, 45];

arr6.reverse();

console.log(arr6);

// Output: [45, 12, 34, 22, 88]
```

6. indexOf()

Kisi element ka first index find karta hai.

```
let arr7 = [90, 225, 134, 172, 405];

console.log(arr7.indexOf(172));

// Output: 3
```

7. slice()

Array ke ek portion ko naya array banakar deta hai.

```
let arr8 = [45, 23, 85, 7, 6, 98];

console.log(arr8.slice(2, 5));

// Output: [85, 7, 6]
```

8. splice()

Array mein add ya remove karne ke liye use hota hai.

```
let arr9 = [89, 65, 23, 78, 98];

// Add karna

arr9.splice(2, 0, 11);

console.log(arr9);

// Output: [89, 65, 11, 23, 78, 98]// Replace karna

let arr10 = [23, 76, 45, 32, 98];

arr10.splice(3, 1, 888);

console.log(arr10);

// Output: [23, 76, 45, 888, 98]
```

map(), filter(), and reduce()

map() Method

map() ka kaam hai har element ko ek nayi shape dena aur ek naya array banakar wapas dena. Yeh original array ko change nahi karta, bas ek naye array mein transformed values de deta hai.

Example:

```
let numbers = [1, 2, 3, 4, 5];

let doubledNumbers = numbers.map((element) => {
  return element * 2;
});
console.log(doubledNumbers);
// Output: [2, 4, 6, 8, 10]
```

Explanation?

1. Ek array hai: [1, 2, 3, 4, 5].
2. map() method har element pe ek function apply karta hai.
   - Yahaan har element ko ***2** kar diya.
3. Result ek naya array ban gaya: [2, 4, 6, 8, 10].

filter() Method

filter() ek array ke andar ghuskar sirf un elements ko chhanta hai jo condition ko pass karte hain. Jo fail karein, unhe ignore karta hai.

Example:

```
let numbers = [1, 2, 3, 4, 5];

let filteredArray = numbers.filter((element) => {
  if (element % 2 === 0) {
    return element;
  }
});
console.log(filteredArray);
// Output: [2, 4]
```

Explanation ?

1. Ek array hai: [1, 2, 3, 4, 5].
2. filter() method har element ko check kar raha hai:
    - Agar element even hai (% 2 === 0), toh nayi array mein add ho jayega.
3. Result nayi array ban gayi: [2, 4] (odd numbers ko ignore kar diya).

reduce() Method

reduce() ek recipe ki tarah kaam karta hai, jo har element ko process karta hai aur ek single output deta hai. Iska use tab hota hai jab sab elements ko combine karna ho.

Example:

```
let numbers = [1, 2, 3, 4, 5];

let sum = numbers.reduce((accumulator, currentValue) => {
  return accumulator + currentValue;
}, 0);
console.log(sum);
// Output: 15
```

Explanation ?

1. Ek array hai: [1, 2, 3, 4, 5].
2. reduce() method ke andar ek function hai:
    - accumulator ka kaam hai result ko store karna.
    - currentValue har baar array ka current element hota hai.
3. Shuru se lekar end tak:
    - 0 + 1 = 1
    - 1 + 2 = 3
    - 3 + 3 = 6
    - 6 + 4 = 10
    - 10 + 5 = 15
4. Final result: 15.

forEach() Method

forEach() ka kaam hai ek array ke har element ko individually process karna.

Bas ye ek loop ki tarah kaam karta hai, lekin zyada readable aur short hota hai.

Dhyan do:

- forEach() kuch return nahi karta, iska kaam sirf process karna hai.

Syntax:

```
array.forEach((element, index, array) => {
  // Code to execute on each element
});
```

- **element**: Current element jo process ho raha hai.
- **index**: (Optional) Current element ka index (position in array).
- **array**: (Optional) Pura array jisme se elements aa rahe hain.

Example 1: Simple Use Case

```
let numbers = [1, 2, 3, 4, 5];
numbers.forEach((element) => {
  console.log(element * 2);
});
// Output:// 2// 4// 6// 8// 10
```

Explanation ?

1. Array [1, 2, 3, 4, 5] ke har element ko forEach() method process kar raha hai.
2. Har element *2 ho raha hai aur result console pe print ho raha hai.
3. Bas, iska kaam khatam! Return kuch nahi karta.

Example 2: Index ka Use

```
let fruits = ["Apple", "Banana", "Cherry"];

fruits.forEach((fruit, index) => {
  console.log(`Index ${index}: ${fruit}`);
});

// Output:
// Index 0: Apple
// Index 1: Banana
// Index 2: Cherry
```

Kya Seekha?

- Har element ke saath uska index bhi print kar sakte ho.
- Useful jab tumhe position track karni ho

JavaScript Object

JavaScript ka object ek real-world cheez ki tarah hota hai jisme properties (state) aur methods (behavior) hote hain.

- **Example:** Ek car ka socho:
    - **Properties:** Color, brand, speed.
    - **Methods:** Start karna, stop karna, accelerate karna.

Key Points about Objects

1. **Multiple Values Store Karna:** Objects ek naam ke andar multiple values store karne ka option dete hain (key-value pairs ke form mein).
2. **Example:** Ek person ke naam, age aur city ko ek saath store karna.
3. **Readable Aur Organized Code:** Variables se better hai kyunki objects ka structure clean aur samajhne layak hota hai.

How to make an Object ?

Object Literal (Sabse Easy Way)

Sabse simple aur zyada use hone wala tarika.

```
let person = {
  id: 1,
  name: "Shaaz",
  age: 23
};
```

```
// Access karna:
console.log(person.name);  // Output: Shaaz
console.log(person["age"]);    // Output: 23
```

Note:

- Dot notation zyada common aur clean lagta hai.
- Bracket notation tab use hota hai jab key dynamic ho ya ek string ho.

Object Constructor (Thoda Detailed Way)

Ek khaali object banao aur properties ko baad mein add karo.

```
let car = new Object();
car.brand = "Toyota";
car.color = "Red";
car.speed = 120;
console.log(car);
// Output: { brand: 'Toyota', color: 'Red', speed: 120 }
```

Constructor Function (Reusability ke Liye Best)

Agar multiple objects create karne hain, toh constructor function ka use karte hain.

```
function Person(name, age, city) {
  this.name = name;
  this.age = age;
  this.city = city;
}
let person1 = new Person("Shaaz", 23, "Delhi");
let person2 = new Person("Aman", 25, "Mumbai");
console.log(person1);
 // Output: { name: 'Shaaz', age: 23, city: 'Delhi' }
console.log(person2);
// Output: { name: 'Aman', age: 25, city: 'Mumbai' }
```

Object Properties ke Saath Kya-Kya Kar Sakte Hain?

Add/Update Properties

```
let laptop = { brand: "HP", price: 50000 };
laptop.ram = "16GB";     // Add new property
laptop.price = 45000;    // Update existing property
console.log(laptop);
// Output: { brand: 'HP', price: 45000, ram: '16GB' }
```

Delete Property

```
delete laptop.ram;
console.log(laptop);
// Output: { brand: 'HP', price: 45000 }
```

Object Methods: Important Cheezein

Object.keys(): Saari keys (property names) ka array laata hai.

```
let car = { brand: "Honda", color: "Blue", speed: 150 };
console.log(Object.keys(car));
// Output: [ 'brand', 'color', 'speed' ]
```

Object.values(): Saari values ka array laata hai.

```
console.log(Object.values(car));
// Output: [ 'Honda', 'Blue', 150 ]
```

Object.entries(): Key-value pairs ko ek nested array ke form mein laata hai.

```
console.log(Object.entries(car));
// Output: [ ['brand', 'Honda'], ['color', 'Blue'], ['speed', 150] ]
```

Object.assign(): Ek object ko dusre object ke saath merge karna ya copy karna.

```
let obj1 = { name: "Shaaz", age: 23 };
let obj2 = { city: "Delhi", profession: "Developer" };
let mergedObj = Object.assign({}, obj1, obj2);
console.log(mergedObj);
// Output: { name: 'Shaaz', age: 23, city: 'Delhi', profession:
'Developer' }
```

Object Immutability: Object.freeze() vs Object.seal()

Object.freeze():

Object ko completely unchangeable bana deta hai.

```
let obj = { name: "Shaaz", age: 23 };
Object.freeze(obj);
obj.age = 25;    // Error (strict mode) or ignored
obj.city = "Delhi"; // Error (strict mode) or ignored
console.log(obj);
// Output: { name: 'Shaaz', age: 23 }
```

Object.seal():

Kewal properties ko update kar sakte ho, naye properties add/delete nahi kar sakte.

```
let obj = { name: "Shaaz", age: 23 };
Object.seal(obj);
obj.age = 25;    // Allowed
obj.city = "Delhi"; // Ignoredconsole.log(obj);
// Output: { name: 'Shaaz', age: 25 }
```

Storing Data in Browser

Local Storage – Permanent Memory for Browser

Samjho ki tumhare paas ek digital locker hai jo browser ke andar hai. Yahan pe tum data save kar sakte ho aur tab tak rak sakte ho jab tak tum manually delete na karo ya browser clear na kare. Ye server se baar-baar data fetch karne ki zaroorat ko kam karta hai, jo performance aur speed dono badhata hai.

Example: Tumhari ek shopping website hai, aur tum chahte ho ki users ke favorite items hamesha store rahein, toh tum Local Storage ka use karoge.

How to Use Local Storage?

1. Storing & Retrieving Simple Values

```
localStorage.setItem("game", "cricket");
console.log(localStorage.getItem("game"));
localStorage.setItem("username", "ShaazAhmad");
localStorage.setItem("username", "ShaazAkhtar");
karnalocalStorage.removeItem("username");
localStorage.clear();
```

Key Point: Local Storage me string format me data store hota hai. Isme sirf same-origin wale pages access kar sakte hain.

2. Storing & Retrieving Objects (Updated Method)

Agar tumhe objects ya arrays store karne hain, toh pehle usko **JSON.stringify()** se string me convert karna padega, aur jab retrieve karoge tab **JSON.parse()** ka use karna padega.

```
let user = {
  name: "Shaaz",
  age: 23,
  city: "Delhi",
};

localStorage.setItem("user", JSON.stringify(user));

let storedUser = JSON.parse(localStorage.getItem("user"));
console.log(storedUser.name); // Output: "Shaaz"
```

Pro Tip: Hamesha JSON.parse() ka use karo jab Local Storage se object ya array retrieve karna ho.

Cookies – Small but Powerful

Cookies ekdum chhoti memory chip ki tarah hoti hain jo browser aur server ke beech ka bridge hoti hain. Ye har request ke saath server ko send hoti hain aur mostly login ya user preferences store karne ke kaam aati hain.

Example: Tumne kisi shopping website pe "Remember Me" option dekha hoga? Ye cookies ka use karke implement hota hai.

How to Use Cookies?

1. Set a Cookie

```
document.cookie = "username=Shaaz; expires=Fri, 31 Dec 2024 23:59:59 GMT";
```

Explanation:

- "username=Shaaz" → Key-Value Pair me store hota hai.
- "expires=..." → Expiry date set ki jati hai, taki data tab tak save rahe.

2. Get a Cookie

Cookies ko directly JavaScript me access kar sakte ho:

```
console.log(document.cookie); // Output: "username=Shaaz"
```

3. Delete a Cookie

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC";
```

Explanation: Expiry date ko past me set kar do, toh browser automatically delete kar dega.

Session Storage – Browser Close Karte Hi Delete Ho Jaata Hai

Ye Local Storage ki tarah hi hai, lekin temporary hai. Jab tak browser tab open hai tab tak ye data store rahta hai, jaise hi close karoge, saara data udd jayega!

Example: Tum ek online exam de rahe ho, aur tumhara current question number store karna hai jab tak exam chal raha hai. Jaise hi tab close karoge, data delete ho jayega.

How to Use Session Storage?

1. Storing & Retrieving Data

```
sessionStorage.setItem("sessionKey", "sessionValue");
console.log(sessionStorage.getItem("sessionKey"));
// Output: "sessionValue"
sessionStorage.clear();
```

Use Case: Session Storage tab use hota hai jab tumhe temporary data store karna ho jo browser band hote hi delete ho jaye.

Session Storage – Browser Close Karte Hi Delete Ho Jaata Hai

Ye Local Storage ki tarah hi hai, lekin temporary hai. Jab tak browser tab open hai tab tak ye data store rahta hai, jaise hi close karoge, saara data udd jayega!

Example: Tum ek online exam de rahe ho, aur tumhara current question number store karna hai jab tak exam chal raha hai. Jaise hi tab close karoge, data delete ho jayega.

How to Use Session Storage?

1. Storing & Retrieving Data

```
sessionStorage.setItem("sessionKey", "sessionValue");
console.log(sessionStorage.getItem("sessionKey"));
// Output: "sessionValue"
sessionStorage.clear();
```

Use Case: Session Storage tab use hota hai jab tumhe temporary data store karna ho jo browser band hote hi delete ho jaye.

Understanding the DOM

DOM Kya Hai?

Maan lo ek shandaar ghar bana rahe ho jisme alag-alag rooms hain (HTML elements).

- HTML pura ghar hai
- Rooms, windows, aur doors uske andar ke elements hain
- JavaScript tumhara haath hai jo in rooms ko modify kar sakta hai

Ek aur example lo, DOM ek tree structure hai, jisme:

- Root node hota hai <html>
- Iske andar branches hoti hain (<head>, <body>, etc.)
- Aur phir chhoti-chhoti leaves hoti hain (<h1>, <p>, <button> etc.)

JavaScript ka kaam hai is tree ke kisi bhi part ko change karna ya access karna!

Example: Basic HTML Structure

Maan lo tumhare paas ek simple web page hai:

```
<!DOCTYPE html>
<HTML>
<head>
<title>DOM Example</title>
</head>
<body>
<h1 id="main-heading">Hello, DOM!</h1>
<p class="text">This is a paragraph.</p>
<button id="myButton">Click Me</button>
<script src="script.js"></script>
</body>
</html>
```

Ye ek basic structure hai, ab JavaScript ke through isko manipulate karna seekhenge!

1. DOM Elements Ko Select Karna

Bhai, sabse pehle kisi bhi element ko pakadna zaroori hai, tabhi toh usme changes kar paoge! Tum different methods ka use kar sakte ho:

Query Selectors

```
// ID se select karna
let heading = document.getElementById("main-heading");
console.log(heading.innerText);
 // Output: "Hello, DOM!"

// Class se select karna
let para = document.querySelector(".text");

// Sirf pehli matching class milegi
console.log(para.innerText);

// Multiple elements select karna
let allParas = document.querySelectorAll(".text");
// Sabhi matching elements milenge
console.log(allParas);
```

Things to remember:

- getElementById() sirf ek element return karta hai.
- querySelector() bhi ek hi element dega, but CSS selectors ka use karta hai.
- querySelectorAll() sabhi matching elements ka NodeList return karega.

2. Event Listeners (User Actions Ko Handle Karna)

Maan lo tum chahte ho ki jab koi button click kare, toh kuch ho.

Toh Event Listeners ka use hota hai!

```
// Button select karna
let button = document.getElementById("myButton");

// Event listener lagana
button.addEventListener("click", function () {
    alert("Button was clicked!");
});
```

Explained:

- addEventListener() ek function hai jo user ke actions ko sunta hai (like click, keypress etc.).
- Jab button pe click hoga, tab alert box dikhayega!

3. DOM Elements Ko Style Karna

Tum JavaScript se directly CSS properties change kar sakte ho!

```
heading.style.color = "blue"; // Text blue ho jayega
heading.style.fontSize = "30px"; // Font size badh jayega
heading.style.backgroundColor = "yellow";
// Yellow background aayega
```

Yeh CSS ka alternative hai, but hamesha classes use karna best practice hota hai!

4.CSS Classes Ko Add, Remove & Toggle Karna

Maan lo tumhe kisi element ka style dynamically change karna hai toh classList use kar sakte ho.

```
// Class add karna
heading.classList.add("highlight");

// Class remove karna
heading.classList.remove("highlight");

// Class toggle karna (Agar hai toh hata dega, agar nahi hai toh add karega)
heading.classList.toggle("highlight");
```

Example Use Case:

Maan lo tum dark mode toggle kar rahe ho, toh tum classList.toggle() use kar sakte ho!

5. Naya Element Banana & Insert Karna

Maan lo tum chahte ho ki JavaScript se ek naya paragraph add ho page pe.

```
// Naya element banana
let newPara = document.createElement("p");

// Usme text dalna
newPara.innerText = "This is a new paragraph added with JavaScript!";

// Isko page pe insert karnadocument.body.appendChild(newPara);
```

Explained:

- createElement() naya HTML element banata hai.
- innerText se uske andar text add kiya.
- appendChild() se HTML ke andar add kar diya!

6. DOM Se Elements Ko Hatana

Agar tum kisi element ko DOM se delete karna chahte ho, toh remove() method ka use karo.

```
// Paragraph select karo
let paraToRemove = document.querySelector(".text");

// Usko hatao
paraToRemove.remove();
```

Explained:

- .remove() directly element hata deta hai page se!

Real-World Example: Dark Mode Toggle

Agar tum dark mode ka button banana chahte ho jo light aur dark mode toggle kare, toh ye code likho:

```
let btn = document.getElementById("myButton");
let body = document.body;

btn.addEventListener("click", function () {
    body.classList.toggle("dark-mode");
});
```

Aur CSS me ek dark mode class bana lo:

```
.dark-mode {
    background-color: black;
    color: white;
}
```

Bas! Ek button dabao aur light/dark mode switch ho jayega!

Asynchronous JavaScript

Synchronous vs Asynchronous Code

Synchronous (Blocking)

Ek kaam jab tak complete nahi hota, tab tak doosra nahi chalega. Sab kuch line-by-line execute hoga.

Example: Maan lo tum ek restaurant me ho aur ek waiter ek time pe sirf ek order lega.

```
console.log("A: Pizza order kiya...");
console.log("B: Waiter pizza bana raha hai...");
console.log("C: Pizza ready!");
```

Problem? Agar ek kaam time leta hai, toh pure script ka execution ruk jata hai.

Asynchronous (Non-Blocking)

Ek task background me chalta rahega, aur baaki kaam rukenge nahi.

Example: Tum buffet system me ho, jisme sab apna khana khud le rahe hain.

```
console.log("A: Pizza order kiya...");
setTimeout(() => {
    console.log("B: Pizza ready ho gaya!");
}, 2000);
console.log("C: Burger order kiya...");
```

Yahan kya ho raha hai?

1. "A: Pizza order kiya..." turant execute ho gaya.
2. setTimeout() ke andar ka code 2 sec ke liye wait karega, lekin tab tak "C: Burger order kiya..." turant execute ho jayega.
3. 2 sec baad "B: Pizza ready ho gaya!" print hoga.

Key Takeaway: JavaScript single-threaded hoti hai, but asynchronous features ka use karke non-blocking kaam kar sakti hai!

How JavaScript Handles Asynchronous Code?

JavaScript me asynchronous kaam karne ke 3 main tarike hain:

1. Callbacks (Purana tareeka)
2. Promises (Better way)
3. Async/Await (Sabse best aur modern)

1 Callbacks – Old Way (Messy)

Callback ek function ke andar ek aur function pass karta hai jo baad me execute hoga.

Example: Callback Hell (Messy Code)

```
function orderPizza(callback) {
    console.log("🍕 Pizza order kiya...");
    setTimeout(() => {
        console.log("✅ Pizza ready!");
        callback(); // Dusra kaam tab hoga jab ye complete hoga
    }, 2000);
}

function eatPizza() {
    console.log("😋 Pizza kha raha hoon...");
}

orderPizza(eatPizza);
```

Problem? Agar multiple callbacks ho gaye toh Callback Hell create ho jata hai!

```
orderPizza(() => {
    makeBurger(() => {
        orderCoffee(() => {
            eatFood();
        });
    });
}); //unreadable code
```

2 Promises – The Better Way

Promise ek commitment ki tarah hai—ya toh resolve (kaam ho gaya) ya reject (kaam fail ho gaya).

Basic Example:

```
let pizzaPromise = new Promise((resolve, reject) => {
    console.log("🍕 Pizza order kiya...");
    setTimeout(() => {
        let success = true; // Try `false` to test rejection
if (success) {
        resolve("✅ Pizza ready!");
    } else {
        reject("❌ Pizza jal gaya!");
    }
  }, 2000);
});

// Promise ko handle karna
pizzaPromise
  .then((message) => console.log(message)) // when resolve
  .catch((error) => console.log(error)); // When Reject
```

Things to remember?

- resolve() → Jab kaam ho jaye.
- reject() → Jab koi error aaye.
- .then() → Jab promise complete ho jaye.
- .catch() → Jab koi error ho.

Advantage: Callback Hell avoid ho gaya, code readable ho gaya!

3 Async/Await – The Best & Cleanest

Promises ka aur bhi easy aur clean version! 🔥

Example:

```
function orderPizza() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve("✅ Pizza ready!");
        }, 2000);
    });
}

async function serveFood() {
    console.log("🍕 Pizza order kiya...");
    let pizza = await orderPizza();
    console.log(pizza);
    console.log("🍔 Burger order kiya...");
}

serveFood();
```

Things to remember ?

- async function ke andar await ka use hota hai jo promise complete hone tak wait karega.
- **.then()** ki zaroorat nahi, code readable ho gaya!

Best Practice: Jab bhi asynchronous code likhna ho, toh async/await ka use karo! ✅

Some More Advanced Promise Concepts

1 Promise.all() – Parallel Execution

Agar multiple promises ek saath chalane ho, toh Promise.all() use karo.

```
let pizza = new Promise((resolve) => setTimeout(() =>
resolve("🍕 Pizza"), 2000));

let burger = new Promise((resolve) => setTimeout(() =>
resolve("🍔 Burger"), 1000));

Promise.all([pizza, burger]).then((foods) => {
   console.log("Sab kuch ready:", foods);
});

// Output (after 2 sec): Sab kuch ready: [ '🍕 Pizza', '🍔
Burger' ]
```

Things to remember?

- Saare promises ek saath start ho gaye.
- Jab sab complete ho gaye, tab ek saath result mil gaya!

2 Promise.race() – Fastest Promise Wins

Agar multiple requests me sirf jo pehle aaye wahi chahiye, toh Promise.race() use karo.

```
let fast = new Promise((resolve) => setTimeout(() =>
resolve(" Fastest!"), 1000));

let slow = new Promise((resolve) => setTimeout(() =>
resolve("🐌 Slow!"), 3000));

Promise.race([fast, slow]).then((winner) => {
   console.log("Winner:", winner);
});

// Output (after 1 sec): Winner: 🚀 Fastest!
```

Things to remember ?

- Jo sabse fast complete hoga, wahi return ho jayega!

Scope in JavaScript

Scope ka matlab hai "ek variable kahaan tak dikh sakta hai aur use ho sakta hai". JavaScript me 3 tareeke ke scopes hote hain:

1️⃣ **Global Scope** – Pura code access kar sakta hai.

2️⃣ **Function Scope** – Sirf ek function ke andar accessible hota hai.

3️⃣ **Block Scope** – {} ke andar defined hota hai (let & const ke saath).

Example:

Maan lo tumhare Ghar (Global Scope) me ek Locker (Variable) hai.

- Tumhare family ke sab log locker use kar sakte hain (Global Scope).
- Tumhare Papa ka ek private locker hai jo sirf unko access hai (Function Scope).
- Tumhare room ke andar ek chhota locker hai jo sirf tum use kar sakte ho (Block Scope).

Chalo, ab isko code se samajhte hain!

1 Global Scope (Sabko Dikhega)

Global Scope me jo variable declare hota hai, usko pura JavaScript code access kar sakta hai.

```
let globalVar = "🌍 I am a global variable";

function showGlobal() {
    console.log(globalVar); // ✅ Accessible
}

showGlobal();
console.log(globalVar); // ✅ Accessible
```

Yahaan globalVar har jagah accessible hai!

2 Function Scope (Sirf Function Ke Andar)

Function ke andar declare kiya gaya variable sirf usi function me access ho sakta hai.

```
function parentFunction() {
    let parentVar = "👴 I am a parent variable";
    console.log(parentVar); // ✅ Accessible inside function
}

parentFunction();
console.log(parentVar); // ❌ ERROR! parentVar is not defined
```

Problem? parentVar function ke bahar access nahi ho sakta!

3 Block Scope (let & const wale)

Agar let ya const use karte ho, toh woh sirf usi {} block ke andar accessible hota hai.

```
{
   let blockVar = "🔒 I exist only inside this block";
   console.log(blockVar); // ✅ Accessible
}
console.log(blockVar); // ❌ ERROR! blockVar is not defined
```

⚡ var block scope follow nahi karta, woh function scope follow karta hai!

```
{
   var blockVar2 = "😈 I can escape the block!";
}
console.log(blockVar2); // ✅ Accessible! (VAR se bacha rehna)
```

👉 Best Practice: Hamesha let aur const ka use karo, var se door raho!

Scope Chain & Lexical Environment

Lexical Environment – Variables Stored Hote Hain

JavaScript me har function apna ek alag memory space create karta hai, jisme uske variables store hote hain. Isko Lexical Environment kehte hain.

Scope Chain – JavaScript Variables Ko Kaise Dhundta Hai

Jab JavaScript kisi variable ko access karne ki koshish karti hai, toh ye pehle apne function me dhoondhti hai, fir parent function me, aur agar waha bhi nahi mila toh global scope me dekhti hai.

```
let globalVar = "🌍 I am global";

function parentFunction() {
    let parentVar = "👨 I am a parent variable";

    function childFunction() {
        let childVar = "👶 I am a child variable";
        console.log(childVar);   // ✅ Available (Child Scope)
        console.log(parentVar);  // ✅ Available (Parent Scope)
        console.log(globalVar);  // ✅ Available (Global Scope)
    }

    childFunction();
}

parentFunction();
```

Things to remember ?

1. **childFunction()** ke andar sabse pehle apne variables check honge.
2. Agar koi variable waha nahi mila, toh ye **parentFunction()** me check karega.
3. Agar waha bhi nahi mila, toh ye global scope me jayega.
4. Agar kisi bhi scope me nahi mila, toh Error aayega.

Parent Function Cannot Access Child Function's Variable

```
function parentFunction() {
    function childFunction() {
        let childVar = "👶 I am a child";
    }
    console.log(childVar); // ❌ ERROR! childVar is not defined
}
```

👉 JavaScript me child function ka variable parent function me access nahi hota!

What is a Closure?

Simple Definition:

Closure ek function ka ability hota hai ki wo apne parent function ke variables ko yaad rakhe, even after parent function execute ho chuka ho.

Ek function + uska lexical scope = Closure

Real-Life Example:

Maan lo tum ek bank locker ka access le rahe ho.

- Locker (parent function) band ho gaya, but tumhare paas ab bhi chabi hai (closure function).
- Tum hamesha us locker ko open kar sakte ho, even after original owner chala gaya!

Example 1: Basic Closure

```
function parent() {
    let parentVar = "👴 I am a parent variable";  // Parent
scope variable
function child() {
        console.log(parentVar);  // Child function ko parentVar
accessible hai
    }

    return child;  // Child function ko return kar rahe hain
}

let childFunction = parent();  // Parent function execute ho
gaya
childFunction();  // Output: "👴 I am a parent variable"
```

Things to remember ?

1. parent() execute ho gaya, lekin parentVar delete nahi hua.
2. childFunction ke paas parentVar ka reference still available hai, isi ko closure kehte hain!
3. Closure ka magic ye hai ki child function ko parent function ke variables access karne ka power milta hai! 🚀

Example 2: Closure with Counter

```
function createCounter() {
    let count = 0; // Private variable
    return function () {
        count++;
        console.log(`Count: ${count}`);
    };
}

let counter = createCounter();
counter(); // Output: Count: 1
counter(); // Output: Count: 2
counter(); // Output: Count: 3
```

Things to remember ?

- count variable parent function ke andar hai, but child function usko modify kar sakta hai!
- Jab bhi counter() call hota hai, count value increase hoti hai, even after parent function execute ho chuka hai!

👉 Yahi closure ka main use hai – private variables banana jo sirf specific functions access kar sakein!

Example 3: Closures in SetTimeout (Async Closures)

```
function delayMessage(msg, delay) {
    setTimeout(function () {
        console.log(msg);
    }, delay);
}

delayMessage("🚀 Hello after 2 seconds!", 2000);
```

Things to remember ?

- setTimeout() ke andar jo function hai, wo closure bana leta hai jo msg ko yaad rakhta hai.
- 2 second baad bhi msg variable exist karta hai, aur print hota hai!

Example 4: Function Factory (Multiple Closures)

Maan lo tum ek function create kar rahe ho jo different discount percentages return kare.

```
function discountCreator(discount) {
    return function (price) {
        return price - (price * discount) / 100;
    };
}

let tenPercentDiscount = discountCreator(10);
let twentyPercentDiscount = discountCreator(20);

console.log(tenPercentDiscount(1000));  // Output: 900
console.log(twentyPercentDiscount(1000)); // Output: 800
```

Things to remember ?

- discountCreator() ek closure bana raha hai jo discount value store karta hai.
- Har function ka apna apna lexical environment hai!

👉 Closures ko smart tareeke se use karke code ko reusable bana sakte ho!

Call, Apply and Bind

Problem Kya Hai?

Normal cases me, jab tum ek object ke andar function banate ho, toh this usi object ko refer karta hai. Lekin kabhi kabhi tum chahte ho ki kisi aur object ka method kisi doosre object ke liye use ho jaye.

💡 **Example:**

Tumhare do dost hain – Shaaz aur Asad.

- Shaaz ke paas ek method hai jo full name print karta hai.
- Ab tumhe chahiye ki yehi method Asad ke liye bhi chale.
- Yahi kaam Call, Apply aur Bind karte hain!

1 Call() Method – Function Ko Turant Call Karo

Working of Call:

- call() function turant execute hota hai.
- this ka reference badalne ke liye call() ka use hota hai.
- Arguments ek-ek karke pass karne hote hain.

Example:

```
const person1 = {
  fName: "Shaaz",
  lName: "Akhtar",
  printFullName: function (hometown, country) {
    console.log(this.fName + " " + this.lName + " from " +
hometown + ", " + country);
  }
};

const person2 = {
  fName: "Asad",
  lName: "Ahmad"
};

// person1 ka method use karke person2 ka full name print karna
person1.printFullName.call(person2, "Bihar", "India");
// Output: Asad Ahmad from Bihar, India
```

Things to remember ?

- **person1.printFullName()** method originally sirf Shaaz ke liye tha.
- **call()** ka use karke yehi method person2 (Asad) ke liye bhi kaam kar raha hai!
- Extra parameters ("Bihar", "India") bhi as arguments pass ho rahe hain! ✅

💡 Call ka simple formula:

```
functionName.call(object, arg1, arg2, ...);
```

2 Apply() Method – Arguments as Array

Apply:

- call() aur apply() me sirf ek difference hai:
  - call() me arguments alag-alag pass hote hain.
  - apply() me arguments ek array ke andar pass hote hain.
- apply() useful hota hai jab arguments pehle se ek array me stored ho.

Example:

```
person1.printFullName.apply(person2, ["Bihar", "India"]);
// Output: Asad Ahmad from Bihar, India
```

Things to remember ?

- call() aur apply() ka result same hai, bas arguments pass karne ka tareeka alag hai!

Apply ka simple formula:

```
functionName.apply(object, [arg1, arg2, ...]);
```

3 Bind() Method – Function Store Karo, Baad Me Call Karo

Bind ka kaam:

- bind() immediately function ko call nahi karta, balki ek naye function ko return karta hai.
- Jab tumhe future me function execute karna ho, tab bind() ka use hota hai!
- Isko ek variable me store karke baad me call kar sakte ho.

Example:

```
const result = person1.printFullName.bind(person2, "Bihar",
"India");


console.log(result);  // Output: [Function: bound
printFullName] (Function return ho gaya)
result();  // Output: Asad Ahmad from Bihar, India
```

Things to remember ?

1. bind() ne function ko turant call nahi kiya, balki ek new function return kiya.
2. result() jab call kiya tab actual execution hua.

Bind ka simple formula:

```
let newFunction = functionName.bind(object, arg1, arg2,
...);
newFunction();
```

Kab Kaunsa Use Karein?

✅ call() – Jab function turant execute karna ho aur arguments normal way me pass karne ho.

✅ apply() – Jab function turant execute karna ho, lekin arguments array ke form me ho.

✅ bind() – Jab function ko future ke liye store karna ho, aur baad me call karna ho.

Real-World Examples

1 Function Borrowing (Call & Apply)

Maan lo ek object me method hai, aur tum doosre object me bina dobara likhe use karna chahte ho.

```
const student = {
    name: "Shaaz",
    greet: function(subject) {
        console.log(`Hello, I am ${this.name} and I love
${subject}!`);
    }
};

const teacher = {
    name: "Rahul Sir"
};

// Student ka method teacher ke liye bhi use ho raha hai!
student.greet.call(teacher, "JavaScript");  // Output: Hello, I
am Rahul Sir and I love JavaScript!
```

2 Using Bind for Event Listeners

Agar tum chahte ho ki ek object ka function event listener me bhi kaam kare, toh bind() useful hota hai.

```
const button = document.getElementById("myButton");

const obj = {
    message: "Button clicked!",
    showMessage: function () {
        console.log(this.message);
    }
};

// Bind ka use karke ensure kiya ki `this` obj ko refer kare!
button.addEventListener("click",
obj.showMessage.bind(obj));
```

3 Partial Functions using Bind

Maan lo tum ek function ko pre-filled arguments ke saath store karna chahte ho.

```
function multiply(a, b) {
    return a * b;
}

const double = multiply.bind(null, 2);  // a = 2 fix ho gaya
console.log(double(5));  // Output: 10
console.log(double(10)); // Output: 20
```

💡 **Things to remember ?**

- bind(null, 2) ne first argument 2 fix kar diya.
- Ab double() jab bhi call hoga, wo 2 × given number return karega!

This in JavaScript

Simple Definition:

👉 JavaScript me this ek special keyword hai jo kisi function ya object ko refer karta hai.

👉 Kis object ko refer karega? Ye uske "execution context" pe depend karta hai!

💡 Ek simple rule yaad rakho:

"this ka value depend karta hai ki function kaise call kiya gaya hai!" ✅

🔥 Different Ways to Use this in JavaScript

1️⃣ this by itself (Global Scope)

Agar this global scope me likha hai, toh ye global object ko refer karega.

- Browser me this = window object.
- Node.js me this = empty object {}.

Example:

```
console.log(this); // Output: window (browser me)
```

Things to remember?

- JavaScript ka sabse bada object window hota hai, jisme console, setTimeout(), aur bahut saari cheezein hoti hain!

2 this inside an Object Method

Agar this kisi object ke method me likha ho, toh wo usi object ko refer karega.

Example:

```
const person = {
  firstName: "Shaaz",
  lastName: "Akhtar",
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
};

console.log(person.fullName()); // Output: "Shaaz Akhtar"
```

Things to remember ?

- this.firstName ka matlab hai ki wo person object ke andar firstName property ko refer kar raha hai!

3 this inside a Function (Alone)

Agar this ek normal function ke andar likha ho, toh global object (window) ko refer karega.

Example:

```
function showThis() {
  console.log(this);
}
showThis(); // Output: window (browser me)
```

Things to remember ?

- Normal function ke andar this window ko point karta hai!

4️⃣ this inside a Function (Strict Mode)

Agar strict mode enable ho, toh this undefined ho jata hai.

👀 Example:

```
"use strict";

function showThisStrict() {

  console.log(this);

}

showThisStrict(); // Output: undefined
```

Things to remember ?

- "use strict" mode me global scope me this undefined ho jata hai!

5️⃣ this inside Arrow Functions (Lexical this)

👉 Arrow functions this ko bind nahi karte, wo apne parent scope se this le lete hain! ✅

Example:

```
const person = {

  firstName: "Shaaz",

  lastName: "Akhtar",

  fullName: function () {

    const arrowFunc = () => {

      console.log(this.firstName + " " + this.lastName);

    };

    arrowFunc();

  }

};

person.fullName(); // Output: "Shaaz Akhtar"
```

Things to remember ?

- Arrow function me this apne parent function (fullName()) se lega!
- Arrow function ka this kabhi change nahi hota!

6 this in Event Listeners

Agar this event listener me use kiya jaye, toh ye us element ko refer karega jispe event laga hai.

Example:

```
document.getElementById("myButton").addEventListener("click", function () {
  console.log(this); // `this` button element ko refer karega
});
```

Things to remember ?

- Event listener ke andar this wo element hoga jispe event laga hai! ✅

7️⃣ this with Call, Apply, and Bind

Agar tum manually this ka reference change karna chahte ho, toh call(), apply(), aur bind() ka use kar sakte ho! 🚀

Example:

```
const person1 = {
  firstName: "Shaaz",
  lastName: "Akhtar",
};


const person2 = {
  firstName: "Asad",
  lastName: "Ahmad",
};


function printName(city, country) {
  console.log(this.firstName + " " + this.lastName + " from " + city + ", " + country);
}
// Call
printName.call(person2, "Bihar", "India"); // Output: "Asad Ahmad from Bihar, India"
// Apply (arguments array me pass hote hain)
printName.apply(person2, ["Bihar", "India"]);


// Bind (function return karta hai, turant call nahi hota)
const newFunc = printName.bind(person2, "Bihar", "India");
newFunc(); // Output: "Asad Ahmad from Bihar, India"
```

Things to remember ?

- call() aur apply() turant function call kar dete hain.
- bind() ek naya function return karta hai, jo baad me call hota hai.

Object-Oriented Programming (OOP)

OOP Kya Hai?

👉 OOP ek programming style hai jo objects ka use karke code likhne ki technique hai.

👉 Objects ek tarah ke "containers" hain jisme data aur methods (functions) hoti hain.

👉 OOP ka goal hota hai code ko zyada organized, reusable aur maintainable banana. ✅

Example:

Maan lo tum ek car manufacturing company ho.

- Har car me kuch common properties hoti hain – color, brand, speed.
- Har car ke paas kuch actions hote hain – start(), stop(), accelerate().
- Har naye model ka ek base structure hota hai, usme naye features add kiye ja sakte hain.
- Yehi OOP ka concept hai!

JavaScript OOP Concepts

JavaScript me OOP ke 4 main pillars hote hain:

1️⃣ **Encapsulation** – Data ko ek object me bundle karna.

2️⃣ **Inheritance** – Ek class dusri class se properties aur methods inherit kar sakti hai.

3️⃣ **Polymorphism** – Ek method ka different objects pe alag behavior ho sakta hai.

4️⃣ **Abstraction** – Unnecessary details hide karna aur sirf necessary cheezein expose karna.

Chalo, har concept ko deep me samajhte hain!

1 Creating Objects (Object Literals)

Sabse simple tareeka objects banane ka {} notation hai.

Example:

```
const person = {
    firstName: "Shaaz",
    lastName: "Akhtar",
    fullName: function () {
        return this.firstName + " " + this.lastName;
    }
};
console.log(person.fullName()); // Output: "Shaaz Akhtar"
```

Things to remember ?

- Object ke andar properties aur methods ho sakte hain.
- Methods ek function hote hain jo object ke andar hote hain.

2 Constructor Functions (Multiple Objects Banane Ka Tareeka)

Agar tumhe bahut saare similar objects banane hain, toh constructor function ka use karo! ✅

Example:

```
function Person(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.fullName = function () {
        return this.firstName + " " + this.lastName;
    };
}

let person1 = new Person("Shaaz", "Akhtar");
let person2 = new Person("Asad", "Ahmad");

console.log(person1.fullName()); // Output: "Shaaz Akhtar"
console.log(person2.fullName()); // Output: "Asad Ahmad"
```

Things to remember?

- new keyword ek naya object banata hai.
- Constructor functions ek tarah ke "**blueprint**" hote hain jisme se multiple objects ban sakte hain. ✅

3 Prototypes (Memory Efficient Objects)

Agar tum constructor function ke andar directly function likhoge, toh har object ek alag copy store karega, jo memory waste karega.

Iska solution hai prototype ka use!

Example:

```
function Person(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
}

// Prototype me method add karna
Person.prototype.fullName = function () {
    return this.firstName + " " + this.lastName;
};

let person1 = new Person("Shaaz", "Akhtar");
console.log(person1.fullName()); // Output: "Shaaz Akhtar"
```

Things to remember ?

- Prototype ka use karke sabhi objects ek hi method share karte hain!
- Yeh memory ko save karta hai aur execution fast hota hai.

4 Inheritance (Ek Class Dusre Se Properties Aur Methods Leta Hai)

👉 Maan lo ek "Animal" class hai, aur tum "Dog" class create karna chahte ho jo Animal ke features le sake!

Example:

```
function Animal(name) {
    this.name = name;
}
Animal.prototype.speak = function () {
    console.log(`${this.name} makes a sound.`);
};
// Dog ko Animal se inherit karna
function Dog(name, breed) {
    Animal.call(this, name); // Animal constructor call karna
    this.breed = breed;
}
// Inheritance set karna
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;
Dog.prototype.bark = function () {
    console.log(`${this.name} barks loudly!`);
};
let dog1 = new Dog("Bruno", "Labrador");
dog1.speak(); // Output: "Bruno makes a sound."
dog1.bark();  // Output: "Bruno barks loudly!"
```

Things to remember ?

- Dog class ne Animal class ke properties aur methods inherit kiye!
- call() ka use karke parent class ka constructor call kiya.

5 Encapsulation (Data Hide Karna)

👉 Encapsulation ka matlab hai data ko private banana taki wo sirf controlled methods se access ho sake.

Example:

```
function BankAccount(owner, balance) {
    let _balance = balance; // Private variable
    this.owner = owner;

    this.getBalance = function () {
        return _balance;
    };

    this.deposit = function (amount) {
        if (amount > 0) _balance += amount;
    };
}

let account = new BankAccount("Shaaz", 5000);
console.log(account.getBalance()); // Output: 5000
account.deposit(2000);
console.log(account.getBalance()); // Output: 7000
console.log(account._balance); // ❌ Undefined (Direct access not allowed)
```

Things to remember?

- Private variables function ke andar define hote hain.
- Unko sirf getter aur setter methods ke through access kiya ja sakta hai.

6 Polymorphism (Ek Method Ka Alag-Alag Behavior)

👉 Same method ka alag-alag classes me different behavior ho sakta hai!

Example:

```
class Animal {
    speak() {
        console.log("Animal makes a sound");
    }
}

class Dog extends Animal {
    speak() {
        console.log("Dog barks");
    }
}

class Cat extends Animal {
    speak() {
        console.log("Cat meows");
    }
}

let myDog = new Dog();
let myCat = new Cat();

myDog.speak(); // Output: "Dog barks"
myCat.speak(); // Output: "Cat meows"
```

Things to remember?

- Different classes me same method ka different behavior ho sakta hai!

JavaScript Modules

JavaScript Modules Kya Hain?

👉 Modules ka matlab hai "Code ko alag-alag files me tod kar organize karna"

👉 Ek module ek file hoti hai jo variables, functions, ya classes ko export ya import karti hai.

👉 Modules ka use karne se code organized, reusable aur maintainable banta hai!

Real-Life Example:

Maan lo tum ek ghar bana rahe ho.

- Bedroom, Kitchen, Bathroom alag-alag hote hain (Modules).
- Har room ka apna alag kaam hota hai – Bedroom me sona, Kitchen me khana banana.
- Poora ghar tabhi complete hoga jab sab rooms connect honge (Application).
- Modules bhi isi tarah alag-alag files me hote hain, jo ek dusre se connect hote hain!

Why Use JavaScript Modules?

✓ **Organized Code** – Ek jagah sab kuch likhne se better hai alag-alag files me likhna.

✓ **Reusability** – Ek baar likho, har jagah use karo!

✓ **Maintainability** – Code ko samajhna aur update karna easy hota hai.

✓ **Performance** – Modules ko lazy load kar sakte ho, jo performance improve karta hai.

Creating & Using JavaScript Modules

1 Creating a Module File (math.js)

Maan lo tum ek math utilities ka module bana rahe ho.

Is file me kuch useful functions define karenge aur export karenge.

Example:

```
// math.js

export function add(a, b) {
    return a + b;
}

export function subtract(a, b) {
    return a - b;
}

export function multiply(a, b) {
    return a * b;
}

export function divide(a, b) {
    if (b !== 0) return a / b;
    else return "Cannot divide by zero!";
}
```

Things to remember ?

- export keyword ka use karke functions ko available banaya.
- Ab ye functions kisi bhi dusre file me import ho sakte hain!

2 Creating Another Module (greeting.js)

Ek greeting module bana rahe hain jo user ko welcome karega.

Example:

```
// greeting.js

export const greetingMessage = "Hello! Welcome to
JavaScript Modules.";


export function greetUser(name) {
    return `Hello, ${name}! Welcome to our website.`;
}
```

Things to remember ?

- export se variables aur functions ko accessible banaya.
- Ab hum inko kisi bhi file me use kar sakte hain!

3 Importing Modules (main.js)

Ab hum math.js aur greeting.js ke functions ko import karenge aur use karenge.

Example:

```
// main.js
import { add, subtract, multiply, divide } from './math.js';
import { greetingMessage, greetUser } from './greeting.js';
console.log(add(5, 3));       // Output: 8
console.log(subtract(10, 4)); // Output: 6
console.log(multiply(6, 7));  // Output: 42
console.log(divide(20, 5));   // Output: 4
console.log(greetingMessage); // Output: "Hello! Welcome to JavaScript Modules."
console.log(greetUser("Shaaz")); // Output: "Hello, Shaaz! Welcome to our website."
```

🧐 Things to remember ?

- import se hum exported functions aur variables use kar sakte hain.
- Curly braces {} ka use sirf named exports ke liye hota hai.

Default Exports – Ek File Ka Main Function Export Karna

Kabhi-kabhi tum ek module me sirf ek hi function ya class export karna chahte ho, uske liye default export ka use hota hai.

1 Creating a Default Export (utils.js)

```
// utils.js
export default function multiply(a, b) {
    return a * b;
}
```

Things to remember ?

- Yahan sirf ek function export ho raha hai, isliye export default use kiya.

2 Importing Default Export (main.js)

```
// main.js
import multiply from './utils.js';
console.log(multiply(4, 5)); // Output: 20
```

Things to remember?

- Default import me {} lagane ki zaroorat nahi hoti!

Dynamic Imports – Jab Zaroorat Ho Tabhi Load Karo

Agar tum modules ko demand pe load karna chahte ho, toh dynamic imports ka use kar sakte ho.

Example (main.js)

```
// main.js
import('./math.js').then(module => {
    console.log(module.add(2, 3)); // Output: 5
});
```

Things to remember ?

- Ye approach page ki speed improve karta hai, kyunki module tabhi load hota hai jab zaroorat ho.

Modules in HTML (Browser)

Agar tum browser me modules use kar rahe ho, toh **<script>** me type="**module**" likhna zaroori hai.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>JavaScript Modules</title>
</head>
<body>
<script type="module" src="main.js"></script>
</body>
</html>
```

Things to remember ?

- type="module" likhna compulsory hai, tabhi import statements kaam karegi.

Error Handling

Error Handling Kya Hai?

👉 Error Handling ka matlab hai code me aane wale unexpected errors ko properly manage karna.

👉 Agar koi error aata hai, toh usko handle karna zaroori hai, nahi toh pura program crash ho sakta hai!

👉 Isse code zyada secure, debug-friendly aur user-friendly hota hai!

Real-Life Example:

Maan lo tum ek car drive kar rahe ho, aur achanak ek bada gadda road pe aa gaya!

- Agar brake nahi lagega, toh tum accident kar doge!
- Brake (Error Handling) laga ke safely gadi control kar sakte ho!
- Agar gadda na hota, tab bhi brake system (finally block) waisa hi kaam karega!

Why Use Error Handling?

✔ Prevent Crashes – Error aane par pura program band na ho.

✔ Debugging Easy – Errors ko track karna aur fix karna easy ho.

✔ Better User Experience – Users ko error messages diye ja sakein instead of sudden crashes.

Basic Error Handling in JavaScript

1 try...catch – Galti Pakdo Aur Handle Karo

try block me jo code error de sakta hai, usko likho.

Agar error aata hai, toh catch block usko handle karega aur program crash hone se bacha lega.

Example:

```
try {
   let result = 5 / 0;
   console.log("Result:", result);  // Output: Infinity (No Error)
   let name = undefined;
   console.log(name.toUpperCase()); // ERROR! Cannot read property 'toUpperCase' of undefined
} catch (error) {
   console.log("Error Occurred:", error.message);
}
```

Things to remember ?

- try me jo error aayega, wo catch block me handle ho jayega!
- Pura program crash nahi hoga, bas error ko gracefully handle kiya jayega.

2 finally – Har Haal Me Chalega!

Agar tumhe koi aisa code likhna hai jo chahe error aaye ya na aaye, hamesha chale, toh finally use karo.

Example:

```
try {
   let x = 10;
   console.log(x.toUpperCase()); //  ERROR!
} catch (error) {
   console.log(" Error Caught:", error.message);
} finally {
   console.log(" Cleanup Complete. This will run no matter what!");
}
```

Things to remember ?

- finally block hamesha chalega, chahe try me error aaye ya na aaye!
- Iska use cleanup operations ke liye hota hai (like closing files, disconnecting DB etc.).

3 throw – Apna Khud Ka Custom Error Banayein

Kabhi-kabhi tumhe apna khud ka custom error throw karna pad sakta hai.

throw ka use karke tum custom error messages generate kar sakte ho.

Example:

```
function checkAge(age) {
    if (age < 18) {
        throw new Error("You must be 18 or older to vote!"); // Custom error throw kiya
    }
    return "✅ You are eligible to vote!";
}
try {
    console.log(checkAge(16)); // Error
} catch (error) {
    console.log("🚨 Error:", error.message);
}
```

Things to remember ?

- throw se hum apni marzi ka custom error generate kar sakte hain.
- Agar condition satisfy nahi hoti, toh error throw ho jata hai!

4 try...catch with async/await

Agar tum asynchronous code use kar rahe ho, toh error handling aur bhi important ho jata hai!

Example:

```
async function fetchData() {
    try {
        let response = await fetch("https://invalid-url.com"); // Invalid URL
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.log("🚨 Error fetching data:", error.message);
    }
}

fetchData();
```

Things to remember ?

- Agar fetch() me koi error aata hai, toh catch block usko handle karega.
- Agar try...catch nahi lagate, toh pura program crash ho jata!

5 Custom Error Classes (Advanced Level)

Agar tum apna custom error type banana chahte ho, toh Error class ko extend kar sakte ho!

Example:

```
class CustomError extends Error {
   constructor(message) {
      super(message);
      this.name = "CustomError";
   }
}

try {
   throw new CustomError("This is a custom error!");
} catch (error) {
   console.log(`${error.name}: ${error.message}`);
}
```

Things to remember ?

- Custom error classes bana sakte hain jo Error class ko extend karein!
- Custom error messages aur handling aur bhi powerful ban sakti hai.

Common Errors in JavaScript

✓ **ReferenceError** – Jab koi undefined variable access karne ki koshish hoti hai.

```
console.log(myVar); //  ReferenceError: myVar is not defined
```

✓ **TypeError** – Jab kisi variable ka expected type match nahi hota.

```
let num = 10;
num.toUpperCase(); //  TypeError: num.toUpperCase is not a function
```

✓ **SyntaxError** – Jab code me koi syntax mistake hoti hai.

```
try {
   eval("alert('Hello'"); // SyntaxError: missing closing bracket
} catch (error) {
   console.log(error.message);
}
```

✓ **RangeError** – Jab value allowed range se bahar ho.

```
let arr = new Array(-1); //  RangeError: Invalid array length
```

setTimeout & setInterval

What are setTimeout and setInterval?

👉 **setTimeout** – Ek function ko kuch der baad ek baar chalata hai.

👉 **setInterval** – Ek function ko baar-baar ek fixed interval pe chalata hai.

Real-Life Example:

Maan lo tum pizza order kar rahe ho. 🍕

- Tumne call kiya aur bola "20 min baad pizza deliver karna" – Ye setTimeout hai.
- Tumne subscription liya jo har 1 mahine baad automatically renew hoga – Ye setInterval hai.

Samajh gaye? Chalo code ke saath explore karte hain!

setTimeout – Function Ko Delay Se Chalana

👉 setTimeout(function, delay) ka use function ko ek specific time ke baad run karne ke liye hota hai.

Example:

```
console.log("🚀 Start...");


setTimeout(() => {
    console.log("⏳ This message will appear after 3 seconds!");
}, 3000);

console.log("✅ Code running...");
```

Things to remember ?

- Pehle "Start..." print hoga, fir "Code running..." print hoga, aur 3 second baad "This message will appear after 3 seconds!" aayega!
- JavaScript asynchronous hai, toh setTimeout background me chalta hai!

clearTimeout – setTimeout Ko Cancel Karna

Agar tum setTimeout ko chalne se pehle hi cancel karna chahte ho, toh clearTimeout() ka use hota hai.

Example:

```
console.log("🚀 Start...");

let timeoutId = setTimeout(() => {
    console.log("⏳ This message won't appear!");
}, 5000);

clearTimeout(timeoutId); // Timeout cancel ho gaya!
console.log("🛑 Timeout cleared!");
```

Things to remember ?

- setTimeout ek ID return karta hai, jisko clearTimeout() se cancel kar sakte hain.
- Yahan 5 second baad message print hona tha, but clearTimeout(timeoutId) se cancel ho gaya!

setInterval – Function Ko Baar-Baar Chalana

👉 setInterval(function, interval) ek function ko repeatedly ek fixed interval ke baad chalata hai.

Example:

```
console.log("🚀 Countdown Started...");


let count = 1;

let intervalId = setInterval(() => {
    console.log(`⏳ Countdown: ${count}`);
    count++;

    if (count > 5) {
        clearInterval(intervalId); // 🛑 Stop after 5 times
        console.log("✅ Countdown Stopped!");
    }
}, 1000);
```

Things to remember ?

- Yeh function har 1 second baad chalega aur count print karega.
- 5 baar print hone ke baad clearInterval() se band ho jayega!

clearInterval – setInterval Ko Rukna

Agar tum kisi interval ko manually stop karna chahte ho, toh clearInterval() ka use hota hai!

Example:

```
let intervalId = setInterval(() => {
    console.log("🔁 This message will keep appearing!");
}, 2000);

setTimeout(() => {
    clearInterval(intervalId); // ❌ Stop interval after 7 seconds
    console.log("🛑 Interval cleared!");
}, 7000);
```

Things to remember ?

- Message har 2 second baad print hoga.
- 7 second ke baad clearInterval(intervalId) se interval stop ho jayega!

Spread & Rest Operators

What are Spread (...) & Rest (...) Operators?

👉 Spread Operator (...) – Ek array ya object ko tod ke individual elements me expand karta hai.

👉 Rest Operator (...) – Multiple values ko ek array me collect karta hai.

Real-Life Example:

Maan lo tum pizza bana rahe ho. 🍕

- Tumhare paas ek ready-made base (spread operator) hai jisme tum cheese, toppings aur sauce mila sakte ho!
- Tumne bahut saare ingredients liye aur ek plate me rakh diye (rest operator).

Spread Operator (...)

👉 Spread Operator ka use arrays aur objects ko expand karne ke liye hota hai.

👉 Ye kisi array ya object ke saare elements ko individual elements me tod deta hai.

1️⃣ Combining Arrays (Array Merge)

Maan lo tumhare paas do arrays hain aur unko merge karna hai.

Example:

```
const nums1 = [1, 2, 3, 4, 5];
const nums2 = [6, 7, 8, 9];

const joinedArray = [...nums1, ...nums2];
console.log(joinedArray);
// Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Things to remember ?

- Spread Operator ne dono arrays ko merge kar diya bina kisi loop ke!

2 Copying an Array (Immutable Copy)

Agar tum array ki ek copy bana ke modify karna chahte ho bina original array ko change kiye, toh spread ka use karo.

Example:

```
const originalArray = [10, 20, 30];
const copiedArray = [...originalArray];


copiedArray.push(40);
console.log(originalArray); // Output: [10, 20, 30] (Original remains same)
console.log(copiedArray);   // Output: [10, 20, 30, 40] (New modified copy)
```

Things to remember ?

- Agar = operator se copy karte, toh dono arrays linked hote.
- Spread operator se ek alag naya array bana jo independent hai.

3 Adding New Properties to Objects

Agar tum ek object me naye properties add karna chahte ho bina original object ko modify kiye, toh spread ka use karo.

Example:

```
const user = { name: "Shaaz", age: 23 };

const updatedUser = { ...user, city: "Mumbai" };

console.log(updatedUser);

// Output: { name: "Shaaz", age: 23, city: "Mumbai" }
```

Things to remember ?

- Original object ko modify kiye bina naye properties add kar sakte hain.

4 Overwriting Properties in Objects

Agar tum kisi object ki existing properties ko overwrite karna chahte ho, toh spread ka use karo.

Example:

```
const user = { name: "Shaaz", age: 23 };

const updatedUser = { ...user, age: 25 }; // Age update ho gayi

console.log(updatedUser);

// Output: { name: "Shaaz", age: 25 }
```

Things to remember ?

- Spread operator se object ki properties ko easily update kar sakte hain.

Rest Operator (...)

👉 Rest Operator ka use multiple values ko ek single array me collect karne ke liye hota hai.

👉 Function parameters ke saath use hota hai jab tumhe unknown number of arguments handle karne ho.

1️⃣ Function Parameters with Rest Operator

Agar tum function me unlimited parameters pass karna chahte ho, toh rest operator use karo!

Example:

```
function addNumbers(...nums) {
   return nums.reduce((sum, num) => sum + num, 0);
}
console.log(addNumbers(1, 2, 3, 4, 5));
// Output: 15
```

Things to remember ?

- ...nums function ke saare arguments ko ek array me collect kar raha hai.
- reduce() ka use karke unko sum kiya gaya.

2 Separating First Element from Rest

Agar tumhe array ke first element ko alag karna ho aur baaki elements ek alag array me store karna ho, toh rest operator use karo!

Example:

```
const [first, ...rest] = [100, 200, 300, 400];

console.log(first); // Output: 100

console.log(rest);  // Output: [200, 300, 400]
```

Things to remember ?

- first variable me array ka first element store ho gaya.
- rest me baaki ke elements store ho gaye.

Real-World Example: Combining Both

Maan lo tum ek food delivery app bana rahe ho jisme pizza ka order process kar rahe ho.

Example:

```
function orderPizza(base, ...toppings) {
   console.log(`🍕 Your ${base} pizza with ${toppings.join(", ")} is ready!`);
}

orderPizza("Thin Crust", "Cheese", "Olives", "Mushrooms", "Chicken");

// Output: 🍕 Your Thin Crust pizza with Cheese, Olives, Mushrooms, Chicken is ready!
```

Things to remember ?

- Base (First argument) alag store ho gaya.
- Baaki toppings ...toppings array me store ho gayi.

API (Data Fetching)

API (Data Fetching) Kya Hota Hai?

👉 API (Application Programming Interface) ka matlab hai do applications ke beech communication.

👉 Server se data lane ke liye hum API calls karte hain.

👉 Data ko hum JSON format me receive karte hain.

Real-Life Example:

Maan lo tum Swiggy/Zomato app use kar rahe ho

- Jab tum restaurant search karte ho, toh app server ko ek request bhejti hai.
- Server wapas JSON format me restaurant ka data bhejta hai.
- App is data ko UI par dikhata hai!

Yehi API Fetching hota hai!

Data Fetching Ke Popular Methods

1️⃣ Fetch API (Modern & Built-in JavaScript method)

2️⃣ Axios (Popular Library, Cleaner Syntax)

3️⃣ AJAX (Old method, still used in some places)

Chalo, in teeno ka deep dive karte hain!

1 Fetch API (Modern JavaScript Way)

fetch() ek built-in JavaScript function hai jo server se data fetch karne ke liye use hota hai.

Example: Fetching Data from API

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json()) // Convert response to JSON
  .then(data => console.log(data)) // Print data
  .catch(error => console.log(" Error:", error));
```

Things to remember ?

- fetch() ek request bhejta hai.
- Pehla .then(response => response.json()) JSON format me convert karta hai.
- Dusra .then(data => console.log(data)) actual data dikhata hai.
- Agar koi error aata hai, toh .catch(error => console.log(error)) usko handle karta hai.

Fetch API with async/await (Better Syntax)

👉 async/await ka use karke code aur clean aur readable ban jata hai!

Example:

```
async function fetchData() {
 try {
 let response = await
fetch("https://jsonplaceholder.typicode.com/posts/1");
 let data = await response.json();
 console.log(data);
 } catch (error) {
 console.log(" Error:", error);
 }
}

fetchData();
```

Things to remember ?

- async function me await ka use karke hum promise ko properly handle kar rahe hain!
- Try-catch block ka use karke error handling bhi ho rahi hai!

2️⃣ Axios (Popular & Cleaner Way)

👉 Axios ek third-party library hai jo Fetch API se zyada easy aur powerful hai!

👉 Ye automatically response ko JSON me convert kar deta hai!

Installation (Browser & Node.js)

Browser me directly use karne ke liye CDN add karo:

```
<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js">
</script>
```

Node.js me install karna ho toh:

```
npm install axios
```

🔥 Axios GET Request

👉 Axios me .then() aur async/await dono kaam karte hain!

Example:

```
axios.get("https://jsonplaceholder.typicode.com/posts/1")
    .then(response => console.log(response.data))
    .catch(error => console.log(" Error:", error));
```

Things to remember ?

- Axios me response automatic JSON me convert hota hai, response.json() ki zaroorat nahi!

Axios GET with async/await

👉 Async/await se code aur clean aur readable ban jata hai.

Example:

```
async function fetchData() {
    try {
        let response = await
axios.get("https://jsonplaceholder.typicode.com/posts/1");
        console.log(response.data);
    } catch (error) {
        console.log(" Error:", error);
    }
}

fetchData();
```

Things to remember ?

- Axios me async/await ka use karke error handling aur readable ho gayi!

Axios POST Request (Server Par Data Bhejna)

👉 Agar tumhe server par data send karna ho, toh axios.post() ka use karo.

Example:

```
async function sendData() {
    try {
        let response = await
axios.post("https://jsonplaceholder.typicode.com/posts", {
            title: "New Post",
            body: "This is a new post",
            userId: 1
        });
        console.log("Data Sent:", response.data);
    } catch (error) {
        console.log("Error:", error);
    }
}

sendData();
```

Things to remember ?

- axios.post() ka use karke hum server par data send kar sakte hain.

3 AJAX (Old Way, Still Used)

👉 AJAX (Asynchronous JavaScript and XML) ek purana tareeka hai data fetch karne ka.

👉 Aaj bhi kuch old applications me iska use hota hai!

Fetching Data Using XMLHttpRequest (Old Method)

```
let xhr = new XMLHttpRequest();
xhr.open("GET",
"https://jsonplaceholder.typicode.com/posts/1", true);
xhr.onreadystatechange = function () {
    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log(JSON.parse(xhr.responseText));
    }
};
xhr.send();
```

Things to remember ?

- Ye purana tareeka hai jo ab modern methods (fetch(), axios) se replace ho gaya hai!

Hoisting

What is Hoisting?

👉 Hoisting ek process hai jisme JavaScript execution ke dauraan variables aur functions ko scope ke top pe move kar deta hai.

👉 Matlab, declare ki gayi cheezein use hone se pehle hi JavaScript ke dimaag me pahuch jaati hain. 🤯

👉 Lekin dikkat yeh hai ki variables ki sirf declaration hoist hoti hai, unki values nahi!

Real-Life Example:

Maan lo tum exam dene gaye ho, aur tum teacher se pehle hi answer sheet maang rahe ho!

- Agar tumne var use kiya hai, toh teacher bolega "Haan, sheet toh le lo, but abhi khaali hai (undefined)".
- Agar tumne let ya const use kiya, toh teacher bolega "Abhi sheet mili hi nahi, error aayega!"
- Agar tum function call kar rahe ho jo function declaration hai, toh teacher bolega "Haan bhai, likho!"

Hoisting with var

👉 var ka declaration hoist hota hai, lekin assignment nahi!

Example:

```
console.log(a); // Output: undefined
var a = 5;
console.log(a); // Output: 5
```

JavaScript Isko Kaise Dekhta Hai?

```
var a;       // Hoisting: Declaration upar chali gayi
console.log(a); // undefined
a = 5;       // Assignment yahan ho rahi hai
console.log(a); // 5
```

Things to remember ?

- var ka declaration hoist hota hai, but value nahi assign hoti.
- Isliye pehla console.log(a); undefined print karega!

Hoisting with let & const

👉 let aur const bhi hoist hote hain, but unhe "**Temporal Dead Zone (TDZ)**" me rakha jata hai.

👉 TDZ ka matlab hai ki jab tak variable declare nahi ho jata, tab tak usko access nahi kar sakte.

Example with let:

```
console.log(a); // ❌ReferenceError: Cannot access 'a' before initialization
let a = 5;
```

JavaScript Isko Kaise Dekhta Hai?

```
// Hoisting ho rahi hai, but Temporal Dead Zone (TDZ) me hai!
let a;
console.log(a); // ❌ ReferenceError
a = 5;
```

Things to remember ?

- let aur const ka declaration hoist hota hai, but TDZ me hota hai, isliye access nahi kar sakte!

Hoisting with Function Declarations

👉 Function declarations hoist hoti hain, matlab function ko define karne se pehle bhi call kar sakte ho!

Example:

```
helloWorld(); //  Works fine
function helloWorld() {
   console.log("Hello World");
}
```

```
helloWorld(); // ✅ Works fine
```

JavaScript Isko Kaise Dekhta Hai?

```
function helloWorld() {  // Function pura ka pura hoist ho gaya!
console.log("Hello World");
}

helloWorld(); // ✅ Works
```

Things to remember ?

- Function declaration hoist hoti hai, isliye isko pehle call karna allowed hai.

Hoisting with Function Expressions (var)

👉 Function expressions var ke saath hoist hoti hain, lekin function ka value hoist nahi hota!

Example:

```
helloWorld(); // ❌ TypeError: helloWorld is not a functionvar
helloWorld = function() {
    console.log("Hello World");
};
```

JavaScript Isko Kaise Dekhta Hai?

```
var helloWorld;  // Hoisting: Bas variable declare hua, function nahi
helloWorld(); // ❌ TypeError: helloWorld is not a function

helloWorld = function() {
    console.log("Hello World");
};
```

Things to remember ?

- Function expression me function ka value hoist nahi hota, sirf var helloWorld; hoist hota hai.

Hoisting with Function Expressions (let & const)

👉 Function expressions jo let ya const ke saath hain, wo bhi hoist hote hain, lekin TDZ me rehte hain!

Example:

```
helloWorld(); // ❌ ReferenceError: Cannot access 'helloWorld' before initialization

let helloWorld = () => {
    console.log("Hello World");
};
```

JavaScript Isko Kaise Dekhta Hai?

```
let helloWorld;  // Hoisting ho rahi hai, but TDZ me hai
helloWorld(); // ❌ ReferenceError

helloWorld = () => {
    console.log("Hello World");
};
```

Things to remember ?

- let aur const ke saath function expressions hoist to hote hain, but access nahi kiya ja sakta jab tak assign na ho.

Memoization

What is Memoization?

👉 Memoization ek optimization technique hai jo results ko cache (memory) me store karta hai.

👉 Agar same input ke saath function dobara call hota hai, toh dubara calculation nahi hoti, balki stored value return hoti hai.

👉 Isse performance improve hoti hai, especially recursive aur expensive calculations ke liye.

Real-Life Example:

Maan lo tum Google Maps pe ek location search karte ho.

- Pehli baar slow milega, kyunki data server se aayega.
- Lekin agar dobara wahi location search karo, toh fatafat result mil jayega!
- Kyuki pehli baar ka result cache (memory) me store ho chuka hai.

Yehi memoization ka kaam hai!

Memoization Without Memoization (Slow Function)

👉 Ek normal function jo slow hai kyunki har baar same calculation karta hai.

Example: Factorial Calculation Without Memoization

```
function factorial(n) {
    if (n === 0) return 1;
    console.log(`Calculating factorial of ${n}`);
    return n * factorial(n - 1);
}

console.log(factorial(5));
console.log(factorial(5)); // Same calculation dobara ho rahi hai
```

Output:

Calculating factorial of 5

Calculating factorial of 4

Calculating factorial of 3

Calculating factorial of 2

Calculating factorial of 1

120

Calculating factorial of 5 ❌ (Same calculation dobara ho rahi hai)

Calculating factorial of 4

Calculating factorial of 3

Calculating factorial of 2

Calculating factorial of 1

120

Problem here?

- Har baar same input ke liye dobara calculation ho rahi hai.
- Performance slow ho rahi hai, agar bada number hua toh CPU busy ho jayega.

Memoization (Fast Function)

👉 Ek function jo computed values ko cache (memory) me store karega, taaki dubara calculate na ho.

Example: Factorial Calculation With Memoization

```
function memoizedFactorial() {
    let cache = {}; // Cache to store previously computed values
return function factorial(n) {
        if (n in cache) {
            console.log(`Fetching from cache: factorial(${n})`);
            return cache[n];
        }

        console.log(`Calculating factorial of ${n}`);
        if (n === 0) return 1;
        cache[n] = n * factorial(n - 1);
        return cache[n];
    };
}

const factorial = memoizedFactorial();

console.log(factorial(5));
console.log(factorial(5)); // Ab ye direct cache se milega
console.log(factorial(6)); // Sirf factorial(6) compute hoga, baaki cache se milega
```

Calculating factorial of 5

Calculating factorial of 4

Calculating factorial of 3

Calculating factorial of 2

Calculating factorial of 1

120

Fetching from cache: factorial(5) ✅

120

Calculating factorial of 6

Fetching from cache: factorial(5) ✅

720

Things to remember ?

- Pehli baar compute hoga, lekin agli baar direct cache se milega!
- Performance bohot improve ho gayi, kyunki duplicate calculations nahi ho rahi!

Memoization for Fibonacci (Slow vs Fast)

👉 Fibonacci series ka calculation recursive hota hai, jo slow ho sakta hai.

👉 Memoization se usko fast kar sakte hain!

Example: Fibonacci Without Memoization (Slow)

```
function fibonacci(n) {
    if (n <= 1) return n;
    console.log(`Calculating fibonacci(${n})`);
    return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log(fibonacci(6)); // Bohot slow hoga
```

✅ Output:

Calculating fibonacci(6)

Calculating fibonacci(5)

Calculating fibonacci(4)

Calculating fibonacci(3)

Calculating fibonacci(2)

Calculating fibonacci(1)

Calculating fibonacci(0)

Calculating fibonacci(2) ❌ (Dubara calculate ho raha hai)

Calculating fibonacci(3) ❌ (Dubara calculate ho raha hai)

Calculating fibonacci(4) ❌ (Dubara calculate ho raha hai)

8

Things to remember ?

- Har baar same calculation ho rahi hai.
- Performance slow ho rahi hai, recursion bohot zyada ho raha hai.

Example: Fibonacci With Memoization (Fast)

```
function memoizedFibonacci() {
    let cache = {};

    return function fibonacci(n) {
        if (n in cache) {
            console.log(`Fetching from cache: fibonacci(${n})`);
            return cache[n];
        }
        console.log(`Calculating fibonacci(${n})`);
        if (n <= 1) return n;
        cache[n] = fibonacci(n - 1) + fibonacci(n - 2);
        return cache[n];
    };
}

const fibonacci = memoizedFibonacci();

console.log(fibonacci(6)); // Pehli baar slow, but cache ban jayega
console.log(fibonacci(6)); // Ab fast, kyunki cache se milega
```

Output:

Calculating fibonacci(6)

Calculating fibonacci(5)

Calculating fibonacci(4)

Calculating fibonacci(3)

Calculating fibonacci(2)

Calculating fibonacci(1)

Calculating fibonacci(0)

Fetching from cache: fibonacci(2) ✅

Fetching from cache: fibonacci(3) ✅

Fetching from cache: fibonacci(4) ✅

8

Fetching from cache: fibonacci(6) ✅

8

Things to remember ?

- Ab sirf pehli baar calculate hoga, baaki cache se fast milega!

Destructuring

What is Destructuring?

👉 Destructuring ek technique hai jo arrays aur objects se values nikalne ko easy banati hai.

👉 Normal tareeke se karne ke bajaye destructuring se ek hi line me kaam ho jata hai.

👉 Kisi bhi value ko alag variable me store karne ke liye super useful hai.

Real-Life Example:

Maan lo tum McDonald's me gaye aur ek Combo Meal order kiya.

- Bina destructuring: Tumko har item ek-ek karke lena padega.
- With destructuring: Tumko ek plate me sab kuch mil jayega, ek line me! ✅

1️⃣ Array Destructuring

👉 Agar tumhe ek array se values extract karni hain, toh destructuring ka use karo. ✅

Example: Normal Array Extraction

```
let colors = ["Red", "Green", "Blue"];

let firstColor = colors[0];

let secondColor = colors[1];

let thirdColor = colors[2];

console.log(firstColor, secondColor, thirdColor);

// Output: Red Green Blue
```

- Har value ke liye alag variable banana pad raha hai!

Array Destructuring (Smart Way)

```
let colors = ["Red", "Green", "Blue"];

let [firstColor, secondColor, thirdColor] = colors;


console.log(firstColor, secondColor, thirdColor);
// Output: Red Green Blue
```

- Ek hi line me array se values extract ho gayi!

2 Object Destructuring

👉 Agar tumhe ek object ke properties extract karni hain, toh destructuring ka use karo.

Example: Normal Object Extraction

```
let person = {
    name: "Shaaz",
    age: 23,
    city: "Mumbai"
};

let name = person.name;
let age = person.age;
let city = person.city;


console.log(name, age, city);
// Output: Shaaz 23 Mumbai
```

- Har property ke liye alag-alag likhna pad raha hai! ❌

Array Destructuring (Smart Way)

```
let colors = ["Red", "Green", "Blue"];

let [firstColor, secondColor, thirdColor] = colors;


console.log(firstColor, secondColor, thirdColor);
// Output: Red Green Blue
```

- Ek hi line me array se values extract ho gayi!

2 Object Destructuring

👉 Agar tumhe ek object ke properties extract karni hain, toh destructuring ka use karo.

Example: Normal Object Extraction

```
let person = {
    name: "Shaaz",
    age: 23,
    city: "Mumbai"
};

let name = person.name;
let age = person.age;
let city = person.city;


console.log(name, age, city);
// Output: Shaaz 23 Mumbai
```

- Har property ke liye alag-alag likhna pad raha hai! ❌

Object Destructuring (Smart Way)

```
let person = {
    name: "Shaaz",
    age: 23,
    city: "Mumbai"
};

let { name, age, city } = person;
console.log(name, age, city);
// Output: Shaaz 23 Mumbai
```

Ek hi line me object se values extract ho gayi!

3 Nested Destructuring

👉 Agar object ya array ke andar aur bhi objects ya arrays hain, toh nested destructuring ka use hota hai.

Example:

```
let user = {
    id: 1,
    personalInfo: {
        firstName: "Shaaz",
        lastName: "Akhtar",
        address: {
            city: "Mumbai",
            country: "India"
        }
    }
};

let { personalInfo: { firstName, lastName, address: { city, country } } } = user;

console.log(firstName, lastName, city, country);
// Output: Shaaz Akhtar Mumbai India
```

- Nested destructuring ka use karke deep properties bhi extract ho gayi!

4️⃣ Destructuring with Default Values

👉 Agar destructuring ke waqt value undefined ho, toh default value set kar sakte ho.

Example:

```
let person = { name: "Shaaz" };
let { name, age = 25 } = person;
console.log(name, age);

// Output: Shaaz 25 (Default value set ho gayi)
```

Agar age nahi mila, toh default 25 set ho gaya!

5️⃣ Destructuring Function Parameters

👉 Agar function ko object pass ho raha hai, toh destructuring se direct properties extract kar sakte ho. ✅

Example:

```
function displayUser({ name, age, city = "Unknown" }) {
    console.log(`${name} is ${age} years old from ${city}.`);
}
let user = { name: "Shaaz", age: 23 };
displayUser(user);

// Output: Shaaz is 23 years old from Unknown.
```

- Object ke properties function ke andar destructure ho rahi hain!

6 Rest Operator in Destructuring

👉 Agar tumhe sirf kuch values chahiye aur baaki values ek alag array/object me store karni ho, toh rest operator (...) ka use kar sakte ho. ✅

Example: Array Destructuring with Rest

```
let numbers = [10, 20, 30, 40, 50];

let [first, second, ...rest] = numbers;

console.log(first, second); // Output: 10 20

console.log(rest);          // Output: [30, 40, 50]
```

- Pehle do values alag nikal li, baaki sab rest me store ho gayi!

Example: Object Destructuring with Rest

```
let person = { name: "Shaaz", age: 23, city: "Mumbai",
country: "India" };

let { name, age, ...rest } = person;

console.log(name, age); // Output: Shaaz 23console.log(rest);
// Output: { city: "Mumbai", country: "India" }
```

- Pehli do properties extract ho gayi, baaki sab rest object me store ho gayi!

Deep Copy vs. Shallow Copy

What is Shallow Copy vs Deep Copy?

👉 Shallow Copy – Bas surface level pe copy hoti hai, original aur copied data linked hote hain.

👉 Deep Copy – Ek bilkul naya object ya array create hota hai, original se koi link nahi hota.

💡 Real-Life Example:

- Shallow Copy: Socho tum WhatsApp pe ek forwarded message bhejte ho. Agar koi usko edit kare, toh original message bhi change ho jata hai!
- Deep Copy: Socho tumne ek message likha aur uska screenshot bhej diya. Ab screenshot change karne ka koi chance nahi!

Shallow Copy (Beware! Linked Data)

👉 Shallow copy sirf reference ko copy karta hai, values ko nahi!

Example: Shallow Copy of Objects

```
let fruit1 = { name: "Apple", color: "Red" };
let fruit2 = fruit1; // ❌ Shallow Copy (Reference Copy Ho Raha Hai)
fruit2.color = "Green"; // Changing fruit2
console.log(fruit1); // Output: { name: "Apple", color: "Green" } ❌ (Original bhi change ho gaya!)
console.log(fruit2); // Output: { name: "Apple", color: "Green" }
```

Problem Kya Hai?

- fruit1 aur fruit2 same memory ko point kar rahe hain.
- Agar fruit2 me kuch change karo, toh fruit1 bhi change ho jata hai!

Shallow Copy Techniques (Beware of Reference)

Agar tum Object.assign() ya Spread Operator (...) ka use kar rahe ho, toh yeh sirf shallow copy bana raha hai.

Example: Shallow Copy Using Object.assign()

```
let person1 = { name: "Shaaz", age: 23 };
let person2 = Object.assign({}, person1); // ❌ Shallow Copy
person2.age = 30;
console.log(person1); // { name: "Shaaz", age: 23 } ✅ (Original safe hai!)
console.log(person2); // { name: "Shaaz", age: 30 }
```

✅ Yahan koi problem nahi thi, kyunki object me nested (deep) values nahi thi!

Deep Copy (Safe Copy, No Links)

👉 Deep copy ek bilkul independent naya object ya array create karta hai.

👉 Agar tum deep copy banao, toh original safe rahta hai.

🔥 Deep Copy Using JSON.parse(JSON.stringify()) (Easy Trick)

```
let user1 = {
    name: "Rahul",
    address: {
        city: "Delhi",
        country: "India"
    }
};

let user2 = JSON.parse(JSON.stringify(user1)); //  Deep Copy
user2.address.city = "Mumbai"; // Changing city in user2
console.log(user1.address.city); // Output: Delhi  (Original safe hai!)
console.log(user2.address.city); // Output: Mumbai
```

Things to remember ?

- JSON.stringify() object ko string me convert karta hai.
- JSON.parse() us string ko dobara object me convert karta hai.
- Isse reference toot jata hai, aur puri tarah se naye object ban jata hai! ✅

🚨 **Downside:**

- Functions ya undefined values remove ho jati hain.

Shallow vs Deep Copy for Arrays

👉 Shallow Copy se arrays linked hote hain, Deep Copy me naye arrays bante hain.

👀 Shallow Copy Example (**Using slice() & concat()**)

```
let arr1 = [1, 2, 3, [4, 5]];
let arr2 = arr1.slice(); //  Shallow Copy
arr2[3][0] = 100;

console.log(arr1); // Output: [1, 2, 3, [100, 5]] ❌ (Original bhi change ho gaya!)
console.log(arr2); // Output: [1, 2, 3, [100, 5]]
```

- Nested arrays linked rahe, toh original bhi change ho gaya!

Deep Copy for Arrays

👉 Agar tum JSON.parse(JSON.stringify()) ya Lodash use karoge, toh deep copy milega.

Example:

```
let arr1 = [1, 2, 3, [4, 5]];
let arr2 = JSON.parse(JSON.stringify(arr1)); // Deep Copy
arr2[3][0] = 100;
console.log(arr1); // Output: [1, 2, 3, [4, 5]]  (Original safe hai!)
console.log(arr2); // Output: [1, 2, 3, [100, 5]]
```

- Ab original array change nahi hua, kyunki deep copy hui thi!

Event Loop & Microtasks

What is the Event Loop?

👉 JavaScript ka event loop ek "traffic controller" ki tarah kaam karta hai.

👉 Ye ensure karta hai ki saare tasks ek sequence me execute ho.

👉 **Main concepts:**

- Call Stack – Functions ko execute karta hai.
- Web APIs – Asynchronous tasks handle karta hai (setTimeout, Promises, etc.)
- Callback Queue – Background se aane wale tasks ko stack me bhejta hai.
- Microtask Queue – Promises aur other microtasks ko priority pe execute karta hai.

Real-Life Example:

Socho tum Domino's me pizza order kar rahe ho.

- **Call Stack** – Tumhara order liya ja raha hai.
- **Web APIs** – Order kitchen me chala gaya (background processing).
- **Callback Queue** – Tumhari order ready hoke queue me aa gayi.
- **Microtask Queue** – VIP customers (Promises, queueMicrotask()) ko fast serve kiya ja raha hai!

JavaScript Execution Flow (Event Loop in Action)

JavaScript ka kaam hota hai synchronous tasks complete karna aur asynchronous tasks ko background me process karna.

Example: Synchronous Code Execution

```
console.log("Start");
console.log("Processing...");
console.log("End");
```

✅ **Output:**

Start
Processing...
End

- Jo bhi synchronous hai wo direct execute hota hai!

Asynchronous Tasks & Event Loop

👉 Asynchronous functions Web APIs me chali jati hain, aur later execute hoti hain.

Example:

```
console.log("Start");
setTimeout(() => {
  console.log("Inside setTimeout");
}, 0);
console.log("End");
```

✅ Output:

Start
End
Inside setTimeout

- setTimeout Web API me chala gaya, aur baad me execute hua.
- Event Loop tab tak stack clear hone ka wait karega, fir callback queue se setTimeout execute karega.

Call Stack, Web APIs, Callback Queue & Microtask Queue (Deep Dive)

👉 Yeh sab milke event loop ko smooth banate hain!

1️⃣ Call Stack

- JavaScript ka main execution area hai.
- Jo bhi function execute hota hai wo Call Stack pe aata hai.

2️⃣ Web APIs

- Browser ke paas setTimeout, DOM Events, Fetch API jese async tasks handle karne ka alag system hota hai.
- Ye tasks Web APIs me process hote hain, na ki Call Stack me.

3️⃣ Callback Queue

- Web APIs me complete hone wale tasks callback queue me jate hain.
- Event loop check karta hai ki Call Stack empty hai ya nahi, fir Callback Queue execute hoti hai.

4️⃣ Microtask Queue

- Promises aur queueMicrotask() yaha store hote hain.
- Microtasks, Callback Queue se pehle execute hote hain.

Microtasks vs Macrotasks (Super Important)

👉 Microtasks zyada priority pe hote hain.

👉 Macrotasks (setTimeout, setInterval) baad me execute hote hain.

Example:

```
console.log("Start");

setTimeout(() => console.log("Inside setTimeout"), 0);

Promise.resolve().then(() => console.log("Inside Promise"));

console.log("End");
```

✅ **Output:**

Start

End

Inside Promise

Inside setTimeout

Things to remember ?

- Promise ka microtask queue zyada priority pe hai, isliye wo setTimeout se pehle execute hoga!

Real-World Example: Event Loop Execution

👉 Chalo ek aur example lete hain jo pura event loop explain karega.

Example:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

✅ Output:

1

4

3

2

Things to remember ?

1. Synchronous Code (console.log("1") & console.log("4")) pehle execute hota hai.
2. Promise Microtask Queue me jata hai (priority high).
3. setTimeout Macrotask Queue me jata hai (priority low).
4. Promise execute hota hai (Microtask first).
5. Baad me setTimeout execute hota hai.

Why Event Loop Matters?

Event Loop ka samajhna zaroori hai kyunki:

✔ Performance Optimize Karne Ke Liye – Code fast aur efficient likhne me madad karta hai.

✔ JavaScript Frameworks (React, Node.js) Samajhne Ke Liye – Asynchronous behavior control karna easy hota hai.

✔ Interview Me JavaScript Ke Tricky Questions Solve Karne Ke Liye!

Debouncing & Throttling

What are Debouncing & Throttling?

👉 **Debouncing** – Multiple calls hone se rokta hai aur last call ko execute karta hai.

👉 **Throttling** – Calls ka gap fix karta hai, taaki frequent execution slow ho sake.

💡 **Real-Life Example:**

- **Debouncing** – Socho tumhara boss tumhe baar-baar kaam ke liye yaad dila raha hai, but tum sirf tab reply karte ho jab last baar yaad dilaaye!
- **Throttling** – Socho tum Instagram pe scroll kar rahe ho aur API sirf har 2 second me ek baar call ho rahi hai, chahe kitna bhi fast scroll karo!

1️⃣ Debouncing in JavaScript

👉 Debouncing ek technique hai jo tabhi function ko execute karti hai jab last call ke baad ek certain time tak koi aur call nahi aayi.

👉 Main use case: Search Bar Suggestions, Window Resize Events, Button Clicks Optimization, etc.

Example: Search Bar Without Debouncing (Problem)

```
function searchAPI(query) {
   console.log(`Searching for: ${query}`);
}
document.getElementById("search").addEventListener("input",
(e) => {
   searchAPI(e.target.value); // Har keypress pe API call ho rahi hai
});
```

Debouncing Solution (Fixing the Issue)

👉 Debouncing se sirf last keystroke ke baad API call hogi, agar user kuch second tak kuch na type kare.

Example:

```
function debounce(func, delay) {
   let timer;
   return function (...args) {
      clearTimeout(timer); // Pehle ka timer clear kar do
      timer = setTimeout(() => {
         func(...args); // Naya timer set karo
      }, delay);
   };
}

// API Call Function
function searchAPI(query) {
   console.log(`Searching for: ${query}`);
}

// Debounced Function
const debouncedSearch = debounce(searchAPI, 500);

// Event Listener
document.getElementById("search").addEventListener("input", (e) =>
{
   debouncedSearch(e.target.value);
});
```

Things to remember ?

- Jab tak user type karta rahega, API call nahi hogi!
- Jab user ruk jayega, tabhi last value ke liye API call hogi!
- Performance improve ho gayi, unnecessary API calls band ho gayi!

2️⃣ Throttling in JavaScript

👉 Throttling ek technique hai jo function ko fix interval me execute karti hai, chahe kitni bhi calls aaye.

👉 Main use case: Scroll Events, Window Resize, Button Spam Prevention, etc.

Example: Scroll Event Without Throttling (Problem)

```
window.addEventListener("scroll", () => {
  console.log("User is scrolling...");
});
```

Problem:

- Jab bhi user scroll karega, har millisecond me event fire ho raha hai!
- Agar heavy calculations ya animations chal rahi ho toh page slow ho sakta hai!

Throttling Solution (Fixing the Issue)

👉 Throttling se sirf har fixed interval me function execute hoga, chahe kitni bhi calls aaye.

Example:

```
function throttle(func, interval) {
    let lastExecuted = 0;
    return function (...args) {
        let now = Date.now();
        if (now - lastExecuted >= interval) {
            lastExecuted = now;
            func(...args);
        }
    };
}

// Scroll Function
function onScroll() {
    console.log("User is scrolling...");
}

// Throttled Function
const throttledScroll = throttle(onScroll, 1000);

// Event Listener
window.addEventListener("scroll", throttledScroll);
```

Things to remember ?

- Ab har second me sirf ek baar function chalega, chahe kitna bhi scroll ho!
- Performance zyada smooth ho gayi, unnecessary function calls block ho gayi!

Currying

What is Currying?

👉 Currying ek technique hai jisme ek function multiple arguments ek saath lene ke bajaye, ek-ek argument lekar return karta hai ek naya function.

👉 Final output tab milega jab saare arguments mil jayenge.

Real-Life Example:

Socho tum Domino's me pizza order kar rahe ho.

- **Normal Function:** Ek hi baar me sab bata diya – "Mujhe ek large pepperoni pizza, Coke aur Fries chahiye!"
- **Curried Function:** Ek-ek step me bola – "Mujhe ek pizza chahiye" → "Size Large" → "Pepperoni" → "Coke bhi chahiye" → "Aur Fries bhi!"

Matlab, ek function multiple calls ke through arguments accept karega!

1️⃣ Normal Function vs Curried Function

👉 Normal function me saare arguments ek saath pass hote hain.

👉 Curried function ek-ek argument lekar function return karta hai.

Example: Normal Function

```
function add(a, b, c) {
   return a + b + c;
}
console.log(add(2, 3, 5)); // Output: 10
```

- Yeh ek normal function hai jo ek hi baar me sab arguments le raha hai.

Example: Curried Function

```
function add(a) {
    return function (b) {
        return function (c) {
            return a + b + c;
        };
    };
}
console.log(add(2)(3)(5)); // Output: 10
```

Things to remember ?

- Yeh function ek-ek argument lekar return ho raha hai.
- Final output tab milega jab teeno arguments mil jayenge!

2️ Converting a Normal Function to a Curried Function

👉 Agar tumhare paas ek normal function hai, toh usko currying me convert kaise karoge?

Example: Using bind()

```
function multiply(a, b) {
    return a * b;
}
let curriedMultiply = multiply.bind(null, 2);
console.log(curriedMultiply(5)); // Output: 10
```

Things to remember ?

- bind(null, 2) se pehla argument lock ho gaya.
- curriedMultiply(5) se b ki value pass ho gayi!

3 Automatic Currying (Using Arrow Functions)

👉 Arrow functions ka use karke currying ko aur easy banaya ja sakta hai!

Example:

```
const add = (a) => (b) => (c) => a + b + c;

console.log(add(2)(3)(5)); // Output: 10
```

Things to remember ?

- Ek hi line me curried function likh diya!

4 Practical Uses of Currying

1. Customizable Function Generation

```
const discount = (rate) => (price) => price - (price * rate / 100);


const tenPercentDiscount = discount(10);

console.log(tenPercentDiscount(200)); // Output: 180

const twentyPercentDiscount = discount(20);

console.log(twentyPercentDiscount(200)); // Output: 160
```

Things to remember ?

- Ek hi function alag-alag discount rates ke liye use kar sakte hain!

2. Event Handlers Optimization

```
const handleEvent = (eventType) => (message) => () => {
    console.log(`${eventType} Event: ${message}`);
};
document.getElementById("clickMe").addEventListener("cli
ck", handleEvent("Click")("Button Clicked"));
```

Things to remember ?

- Alag-alag events ke liye ek hi handler ka use kiya!

3. API Call Optimization

```
const fetchData = (baseURL) => (endpoint) => (id) => {
    return fetch(`${baseURL}/${endpoint}/${id}`)
        .then(response => response.json())
        .then(data => console.log(data));
};

const getUser =
fetchData("https://jsonplaceholder.typicode.com")
("users");

getUser(1); // Fetches user with ID 1
getUser(2); // Fetches user with ID 2
```

Things to remember ?

- Ek hi API base URL ko reuse karna easy ho gaya!